СИСТЕМА «СЕМЕЙНАЯ БУХГАЛТЕРИЯ» (FAMILY BUDGET)

ЧАСТНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ

СБ.62.01.11.26.ЧТЗ

На 70 листах

2026

Аннотация

В данном документе приведен полный перечень функциональных, нефункциональных и прочих требований для разработки и развития Системы учёта личного и семейного бюджета «Семейная бухгалтерия» (Family Budget), далее – Система.

Настоящее Частное техническое задание (ЧТЗ) является основным документом, определяющим требования к Системе и порядок её разработки, в соответствии с которым проводятся разработка, проверка и приемка версий Системы. ЧТЗ описывает состояние Системы версии 0.1 и требования к этапам развития до версии 1.0.

Документ предназначен для владельца проекта, разработчиков (в том числе ИИ-ассистентов, выполняющих задачи разработки) и лиц, осуществляющих проверку и приемку версий Системы.

Содержание

Перечень терминов, сокращений и обозначений....6
1 Общие сведения....9
1.1 Полное наименование Системы и ее условное обозначение....9
1.2 Наименование работ....9
1.3 Сокращенное наименование работ....9
1.4 Заказчик....9
1.5 Пользователи Системы....9
1.6 Подрядчик....9
1.7 Основания для выполнения работ....9
1.8 Плановые сроки начала и окончания работ....10
1.9 Сведения об источнике и порядке финансирования работ....10
1.10 Порядок оформления и предъявления Заказчику результатов работ....10
2 Назначение и цели создания Системы....11
2.1 Назначение Системы....11
2.2 Цели выполнения работ....11
3 Характеристики объекта автоматизации....13
3.1 Краткие сведения об объекте автоматизации....13
3.2 Текущее состояние объекта автоматизации....13
3.2.1 Общие сведения....13
3.2.2 Дополнительные сведения....13
3.2.3 Сведения об условиях эксплуатации объекта автоматизации и характеристиках окружающей среды....14
3.3 Этапы развития Системы....14
4 Требования к выполнению работ по разработке Системы....15
4.1 Общие принципы создания Системы....15
4.2 Требования к Системе в целом....15
4.2.1 Требования к структуре и функционированию Системы....15
4.2.2 Требования к локализации модулей Системы....18
4.3 Требования к функциям модулей Системы....19
4.3.1 Требования к функциям модуля «Обзор» (бюджет)....20
4.3.2 Требования к функциям модуля «Операции»....22
4.3.3 Требования к функциям модуля «Календарь» (планирование)....29
4.3.4 Требования к функциям модуля «Лимиты»....32
4.3.5 Общие требования к справочникам....34
4.3.6 Требования к функциям модуля «Категории»....34
4.3.7 Требования к функциям модуля «Счета»....35
4.3.8 Требования к функциям модуля «Места»....36

4.3.9 Требования к функциям модуля «Люди»....37
4.3.10 Требования к функциям модуля «Валюты»....38
4.3.11 Требования к функциям модуля «Курсы валют»....39
4.3.12 Требования к функциям модуля «Проекты и участники»....40
4.3.13 Требования к функциям модуля «Профиль»....42
4.3.14 Требования к функциям модуля «Ассистенты (МСР)»....43
4.3.15 Требования к функциям модуля «Сайт» (лендинг и правовые страницы)....46
4.3.16 Требования к ролевой модели Системы....47
4.4 Требования к видам обеспечения....48
4.4.1 Требования к математическому обеспечению....48
4.4.2 Требования к информационному обеспечению....49
4.4.3 Требования к лингвистическому обеспечению....51
4.4.4 Требования к программному обеспечению....52
4.4.5 Требования к техническому обеспечению....53
4.4.6 Требования к метрологическому обеспечению Системы....53
4.4.7 Требования к телекоммуникационному обеспечению Системы....53
4.4.8 Требования к методическому обеспечению Системы....54
4.5 Общие технические требования....54
4.5.1 Требования к численности и квалификации персонала Системы и режиму его работы 54
4.5.2 Требования к показателям назначения....54
4.5.3 Требования к надежности....56
4.5.4 Требования к безопасности....56
4.5.5 Требования к эргономике и технической эстетике....56
4.5.6 Требования к защите информации от несанкционированного доступа....58
4.5.7 Требования по сохранности информации при авариях....59
4.5.8 Требования к патентной чистоте и лицензионному программному обеспечению....59
4.5.9 Требования по стандартизации и унификации....59
4.5.10 Требования к разработке....59
4.5.11 Требования к регистрации событий....60
5 Состав и содержание работ....61
5.1 Порядок контроля и приемки выполнения работ....62
5.1.1 Виды, состав, объем и методы испытаний Системы и ее составных частей...62
5.1.2 Общие требования к приемке работ....63
5.1.3 Порядок выполнения доработок и устранения ошибок, выявленных в процессе испытаний и эксплуатации....64
5.1.4 Сведения о порядке учета и контроля выполняемых работ....64

6 Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу Системы в действие....65
6.1 Развертывание и конфигурирование....65
6.2 Приведение поступающей в Систему информации к виду, пригодному для обработки с помощью ЭВМ....65
6.3 Изменения, которые необходимо осуществить в объекте автоматизации....65
6.3.1 Создание условий функционирования объекта автоматизации....65
6.3.2 Создание необходимых для функционирования Системы подразделений и служб 65
6.3.3 Сроки и порядок обучения пользователей....66
7 Требования к документированию....67
7.1 Состав документации....67
8 Требования к информационной безопасности....68
8.1 Требования к реализации ролевой модели....68
8.2 Требования к реализации парольной политики....68
8.3 Требования к механизмам идентификации и аутентификации....68
8.4 Требования к защите каналов передачи данных....68
8.5 Требования к аудиту....68
8.5.1 Состав регистрируемых событий....68
8.5.2 Разграничение доступа к данным аудита....69
8.5.3 Функциональные требования к экранной форме....69
8.5.4 Хранение данных аудита....69
8.6 Требования к защите приложений (контейнеризация)....69
8.7 Требования к защите API и MCP....69
8.8 Требования к защите персональных данных....70

Перечень терминов, сокращений и обозначений

Используемые в настоящем документе термины и основные понятия области автоматизированных систем определены в ГОСТ Р 59853-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Термины и определения». Термины предметной области и сокращения приведены в таблице ниже.

Таблица – Используемые в настоящем документе термины, сокращения и обозначения

Сокращение/термин	Определение
2FA	Two-factor authentication – двухфакторная аутентификация (второй фактор входа: код по email, passkey)
API	Application programming interface – программный интерфейс приложения
CI/CD	Continuous Integration / Continuous Delivery – непрерывная интеграция и поставка: автоматическая проверка и сборка каждого изменения
CSP	Content Security Policy – политика безопасности содержимого веб-страницы
CSRF	Cross-Site Request Forgery – подделка межсайтовых запросов
GHCR	GitHub Container Registry – реестр образов контейнеров GitHub
HSTS	HTTP Strict Transport Security – механизм принудительного использования HTTPS
HTTP, HTTPS	Hypertext Transfer Protocol (Secure) – протокол прикладного уровня передачи данных (с шифрованием TLS)
htmx	Библиотека частичного обновления веб-страниц без JS-фреймворка
i18n	Internationalization – механизм многоязычности интерфейса и данных
IDOR	Insecure Direct Object Reference – доступ к чужим данным по прямой ссылке на объект
ISO 4217	Международный стандарт трёхбуквенных кодов валют (THB, RUB, USD)
JSON	JavaScript Object Notation – текстовый формат обмена данными
MCP	Model Context Protocol – протокол подключения ИИ-ассистентов (Claude, ChatGPT) к внешним системам
OIDC	OpenID Connect – протокол аутентификации поверх OAuth 2.0
OWASP ASVS	Application Security Verification Standard – стандарт проверки безопасности веб-приложений OWASP
PKCE	Proof Key for Code Exchange – защита обмена кода авторизации OAuth 2.0
Rauthy	Сервер входа (OpenID Connect) с открытым исходным кодом, используемый Системой для регистрации, входа и второго фактора
REST	Representational State Transfer – архитектурный стиль программного интерфейса поверх HTTP
SMTP	Simple Mail Transfer Protocol – протокол отправки электронной почты

Сокращение/термин	Определение
SQL	Structured Query Language – язык запросов к реляционной СУБД
TLS	Transport Layer Security – протокол защиты транспортного уровня
UI	User interface – пользовательский интерфейс
URL	Uniform Resource Locator – универсальный указатель ресурса
UTC	Всемирное координированное время
VDS	Virtual Dedicated Server – виртуальный выделенный сервер
WCAG	Web Content Accessibility Guidelines – руководство по доступности веб-контента
Ассистент	ИИ-ассистент (Claude, ChatGPT), подключённый к проекту пользователя по MCP и действующий от его имени в пределах выданных прав
Базовая валюта	Валюта проекта, в которой считаются бюджет, лимиты и итоги; задаётся при создании проекта и не меняется
БД	База данных
Быстрый ввод	Диалог добавления расхода, дохода или перевода поверх любой страницы Системы
Владелец (OWNER)	Роль участника проекта с полными правами, включая управление участниками и удаление проекта
Внешняя запись	Связь операции с записью внешнего источника (например, SMS банка): поставщик и идентификатор записи
Вхождение	Конкретная дата плановой операции со статусом «Ожидается», «Просрочено», «Оплачено» или «Пропущено»
Выписка	Список операций по одному счёту с нарастающим остатком
Гость	Пользователь демо-входа «Посмотреть демо» без регистрации, роль VIEWER в демо-проекте
Двойная запись	Способ учёта, при котором сумма проводок операции по каждой валюте равна нулю
Держатель счёта	Участник проекта, которому принадлежит счёт (карта); сведения справочные, доступ не ограничивают
«Для кого»	Получатель покупки (позиции чека): «Для всех», «Для себя» или человек из справочника «Люди»
Журнал безопасности	Журнал событий доступа и изменения прав в Системе
Заказчик	Владелец проекта «Семейная бухгалтерия»
ИБ	Информационная безопасность
Категория	Узел дерева статей расходов или доходов (до 5 уровней)
Код	Неизменяемый идентификатор записи справочника: латинские буквы, цифры и «_», до 40 знаков (например, CARD_KBANK)
Курсовая разница	Результат проводок системного счёта «Обмен валют» за период в базовой валюте
Лимит	Предел расходов категории и всего её поддерева за неделю, месяц или год в базовой валюте
Личный API-токен	Секретный ключ вида fbt_… для доступа к REST API и MCP без браузера: один проект, права, срок действия
Место	Магазин, кафе, рынок или иное место оплаты; подставляет категорию и счёт по умолчанию
Наблюдатель (VIEWER)	Роль участника проекта с правом только просмотра

Сокращение/термин	Определение
Операция (транзакция)	Учётное событие целиком: дата, тип, описание и две или более проводки
ОС	Операционная система
Плановая операция (план)	Шаблон ожидаемого расхода, дохода или перевода с правилом повторения
ПО	Программное обеспечение
Подрядчик	Разработчик Системы
Позиция чека	Строка операции расхода или дохода: категория, сумма, название, количество, цена, скидка, «Для кого»
Проводка	Строка операции: счёт, категория или системный счёт, валюта и сумма со знаком
Проект	Бюджет, которому принадлежат справочники, операции, планы и лимиты: личный (PERSONAL) или семейный (FAMILY)
Редактор (EDITOR)	Роль участника проекта с правом ввода и изменения данных и справочников
Сверка остатка	Приведение остатка счёта в Системе к фактическому остатку банка разницей-операцией
Система	Система учёта личного и семейного бюджета «Семейная бухгалтерия»
Системный счёт	Служебная сторона проводок вне справочников: «Обмен валют» (EXCHANGE), «Начальные остатки» (OPENING)
Справочник	Перечень записей проекта: валюты, курсы, счета, категории, места, люди
СУБД	Система управления базами данных
Участник	Пользователь, состоящий в проекте с ролью OWNER, EDITOR или VIEWER
Человек	Запись справочника «Люди»: участник проекта или лицо без аккаунта (ребёнок, няня)
ЧТЗ	Частное техническое задание

1 Общие сведения

1.1 Полное наименование Системы и ее условное обозначение

Полное наименование системы: Система учёта личного и семейного бюджета «Семейная бухгалтерия» (Family Budget).

Условное обозначение системы: Система.

1.2 Наименование работ

Выполнение работ по разработке и развитию Системы учёта личного и семейного бюджета «Семейная бухгалтерия».

1.3 Сокращенное наименование работ

Разработка Системы «Семейная бухгалтерия».

1.4 Заказчик

Владелец проекта «Семейная бухгалтерия».

Адрес репозитория проекта: https://github.com/Yolshin195/family-budget.

Адрес для переписки: задачи (Issues) и запросы на изменение (Pull Requests) репозитория проекта.

1.5 Пользователи Системы

Пользователями Системы являются:

- члены семьи (домохозяйства), ведущие общий семейный бюджет, и лица, ведущие личный бюджет;
- гости сайта, знакомящиеся с Системой в демо-проекте без регистрации;
- ИИ-ассистенты (Claude, ChatGPT), подключённые пользователем к своему проекту и действующие от его имени в пределах выданных прав;
- внешние клиенты пользователя без браузера (программы синхронизации SMS банка, скрипты), работающие по личному API-токену.

Обслуживающим персоналом Системы является владелец проекта, выполняющий функции администратора сервера, СУБД и сервера входа Rauthy.

1.6 Подрядчик

Разработчик Системы (далее – Подрядчик). Разработка ведётся владельцем проекта с применением ИИ-ассистента Claude Code по правилам, закреплённым в репозитории проекта (файл CLAUDE.md).

1.7 Основания для выполнения работ

Основанием для выполнения работ являются:

- техническое задание «Семейная бухгалтерия», версия 3 от 13.10.2026;
- дорожная карта развития Системы (аудит кода от 17.10.2026, обновление от 21.10.2026);
- техническое задание на реализацию группы 1 «Карточка операции: позиции чека, время, места, «Для кого»»;
- техническое задание «Платформы: Telegram, мобильное и десктоп-приложение, офлайн»;
- план MCP-коннектора «Семейной бухгалтерии» для Claude.

При выполнении работ должны учитываться:

- ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы» – в части состава и содержания настоящего документа;
- OWASP Application Security Verification Standard, уровень 2 – в части требований к безопасности;
- WCAG 2.1, уровень AA – в части доступности интерфейса;
- ISO 4217 – в части кодов валют;
- Регламент (ЕС) 2016/679 (GDPR) и Закон Королевства Таиланд о защите персональных данных (PDPA) – в части удаления аккаунта и персональных данных пользователя.

1.8 Плановые сроки начала и окончания работ

Начало работ: 09.10.2026.

Окончание работ: выпуск версии Системы 1.0.

Сроки начала и окончания этапов работ определяются при планировании каждого этапа на основе ориентировочных сроков, приведенных в разделе 5 настоящего ЧТЗ.

1.9 Сведения об источнике и порядке финансирования работ

Финансирование работ осуществляется за счет собственных средств Заказчика. Договор на выполнение работ не заключается.

1.10 Порядок оформления и предъявления Заказчику результатов работ

Результаты работ предъявляются Заказчику в виде запросов на изменение (Pull Request) в основную ветку репозитория проекта. Каждый запрос на изменение должен содержать:

- описание изменения на русском языке;
- автоматические тесты на изменённое поведение;
- переводы всех видимых пользователю строк на русский, английский и тайский языки;
- запись в «Что нового» к следующему выпуску (три файла в site/content/releases/next/) либо строку «Changelog: none», если изменение пользователю незаметно;
- успешное прохождение автоматических проверок CI.

Выпуск версии оформляется изменением номера версии в Cargo.toml, статьёй о выпуске на сайте и публикацией образа Системы в GHCR с тегом версии. Требования к составу и оформлению результатов работ определены разделами 5, 6 и 7 ЧТЗ.

2 Назначение и цели создания Системы

2.1 Назначение Системы

Назначением Системы является ведение личного и семейного учёта денежных средств нескольких людей: учёт счетов в разных валютах и операций по двойной записи, раскладка чеков по категориям, планирование регулярных платежей и контроль расходов по лимитам дерева категорий.

Система позволяет:

- вести общий (семейный) и раздельные (личные) бюджеты без дублирования справочников;
- записывать расход за время не более 5 секунд с любой страницы, а также голосом и по фото чека через ИИ-ассистента;
- видеть остатки счетов, которые сходятся с банком, и бюджет недели, месяца и года в базовой валюте;
- работать на русском, английском и тайском языках с данными, введёнными на любом из них.

Система должна обеспечить автоматизацию процессов:

- Учёт операций: расходы с позициями чека, доходы, переводы и обмен валют между счетами одной операцией, начальные остатки, выписка и сверка остатка счёта.
- Планирование: плановые расходы, доходы и переводы с правилами повторения, календарь платежей, подтверждение оплаты и пропуск платежа.
- Контроль расходов: лимиты по дереву категорий, бюджет «план – факт – прогноз – доступно», предупреждения о приближении к лимиту и превышении.
- Совместная работа: проекты с участниками и ролями, приглашение по ссылке, требование второго фактора от участников.
- Ввод через ассистентов и внешние клиенты: MCP-сервер для Claude и ChatGPT, REST API с личными API-токенами, приём операций из SMS банка без дублей.
- Ведение справочников: валюты и курсы, счета, категории, места, люди – с названиями на трёх языках.

2.2 Цели выполнения работ

Целью выполнения работ является создание Системы, которой семья доверяет учёт своих денег: цифры Системы совпадают с банком, а ввод операций занимает меньше времени, чем ведение таблиц вручную.

Цели автоматизации:

- Достоверность: остатки счетов, бюджет и отчёты всегда сходятся за счёт двойной записи; ни одно число не показывается неверно без предупреждения.
- Скорость ввода: типичный расход записывается за 3 действия (сумма, категория, Enter) не дольше 5 секунд.
- Совместность: несколько людей ведут общий и личные бюджеты в одной Системе, каждый видит только свои проекты.
- Предсказуемость: регулярные платежи видны в календаре заранее, бюджет месяца виден целиком до первой оплаты.
- Безопасность: защита по OWASP ASVS уровня 2, двухфакторная аутентификация, изоляция данных проектов.

Критериями достижения поставленных целей являются:

- Корректность учёта: сумма проводок каждой операции по каждой валюте равна нулю; остаток счёта одинаков в выписке, справочнике счетов и «Обзоре».
- Скорость: расход добавляется с любой страницы за время не более 5 секунд без перехода на другую страницу.
- Отсутствие известных ошибок в отчётах: новые функции не выпускаются, пока в отчётах есть известные ошибки.
- Удержание: семья продолжает вести бюджет через 30 дней после начала использования; растёт доля операций, внесённых через ассистентов и импорт.
- Полнота курсов: для каждой активной валюты нет ни одного дня без курса к базовой валюте.

Система должна позволять решать следующие задачи:

- раскладка одной покупки (чека) на позиции с разными категориями и получателями;
- перевод и обмен валюты между счетами одной операцией с вычислением курса из сумм;
- контроль расходов по любой ветке дерева категорий с учётом лимитов предков;
- работа с телефона и компьютера, в светлой и тёмной теме, с клавиатуры;
- приём операций от внешних клиентов и ассистентов без дублей и без права удаления.

3 Характеристики объекта автоматизации

3.1 Краткие сведения об объекте автоматизации

Объектом автоматизации Системы является деятельность семьи (домохозяйства) и отдельных людей по учёту доходов, расходов, остатков на счетах в разных валютах и по планированию регулярных платежей.

В рамках создания Системы автоматизируются следующие процессы:

- Учёт операций по счетам наличных, банковских карт, банковских и накопительных счетов в нескольких валютах.
- Раскладка чеков по категориям, местам покупки и получателям.
- Планирование и контроль регулярных платежей.
- Контроль расходов по лимитам и формирование бюджета за неделю, месяц и год.
- Совместное ведение бюджета участниками семьи с разграничением прав.

Система должна обеспечивать:

- Сбалансированность каждой операции по двойной записи.
- Изоляцию данных проектов: пользователь видит только проекты, в которых состоит.
- Работу на трёх языках: интерфейс, справочники и описания операций на русском, английском и тайском.
- Интеграцию с сервером входа Rauthy и ИИ-ассистентами по MCP.

3.2 Текущее состояние объекта автоматизации

3.2.1 Общие сведения

На момент составления ЧТЗ выпущена версия Системы 0.1.0 (17.10.2026). Реализованы: проекты с участниками и ролями, справочники с переводами, операции по двойной записи с позициями чека, переводы и обмен валют, лимиты по дереву, календарь плановых платежей, бюджет недели, месяца и года, быстрый ввод, вход через Rauthy, MCP-сервер для Claude, лендинг и сайт с блогом. Подготовлены к следующему выпуску: справочники «Места» и «Люди», позиции чека с ценой, количеством и скидкой, отметка операций из SMS банка, подключение ChatGPT, приглашение участников только по ссылке, сообщения об ошибках формы операции на языке интерфейса.

Система эксплуатируется на одном VDS: приложение, СУБД PostgreSQL и сервер входа Rauthy в контейнерах Docker за обратным прокси Caddy с HTTPS. Сайт с блогом публикуется отдельно на GitHub Pages и закрыт от индексации.

3.2.2 Дополнительные сведения

По результатам аудита кода от 17.10.2026 устранены ошибки, при которых отчёты показывали неверные числа (пересчёт лимитов по периодам, смена типа категории при наличии операций, суммирование валют без курса, правка правила плана задним числом, проверка оплаты плана, точность валюты), и закрыты найденные уязвимости (токены без второго фактора, добавление в проект без согласия, ограничения гостевого входа).

Остаются риски, устраняемые на этапе 1 развития (раздел 5): недостаточное покрытие тестами SQL-адаптеров и триггера двойной записи на реальной СУБД, хранение состояния «Отменить» и лимитов запросов в памяти процесса, определение «сегодня» по часовому поясу сервера, отсутствие проверки готовности СУБД и корректного завершения при обновлении.

3.2.3 Сведения об условиях эксплуатации объекта автоматизации и характеристиках окружающей среды

Серверная часть Системы эксплуатируется на арендованном VDS с ОС Linux в среде контейнеризации Docker (Podman – для локальной разработки). Требования к климатическим условиям, электропитанию и размещению оборудования обеспечиваются поставщиком VDS.

Клиентская часть Системы используется в веб-браузерах на персональных компьютерах и мобильных телефонах пользователей с доступом в сеть Интернет. ИИ-ассистенты подключаются к Системе из облачных сервисов их поставщиков (claude.ai, приложения Claude, ChatGPT) по сети Интернет.

3.3 Этапы развития Системы

Система разрабатывается поэтапно, от версии 0.1 до версии 1.0. Цели этапов:

- Этап 0 «Цифрам можно верить» (версия 0.1.x) – ни одного тихо неверного числа в отчётах, закрытые найденные уязвимости.
- Этап 1 «Фундамент для роста» (версия 0.2) – тесты на реальной СУБД, наблюдаемость, состояние в базе, ядро без сервера, единая модель прав клиентов, приём операций без дублей.
- Этап 2 «Удобно каждый день» (версия 0.3) – ошибки на трёх языках, автоматические курсы, поиск и фильтры операций, мастер первого запуска.
- Этап 3 «Приложение работает за вас» (версия 0.4) – импорт и «Входящие», Telegram-бот, напоминания, экспорт.
- Этап 4 «Везде и без интернета» (версия 0.6) – синхронизация и приложения для iPhone, Android и компьютера с работой без сети.
- Этап 5 «Аналитика и семья» (версия 0.7) – графики, покупки в другой валюте, кредитки и долги, цели накоплений.
- Этап 6 «Запуск для людей» (версия 1.0) – собственный сервис входа, сайт на основном домене, формальная проверка безопасности и доступности.

Состав работ этапов приведен в разделе 5. Требования к функциям, реализуемым на этапах 1–6, приведены в подразделах «Требования к развитию модуля» раздела 4.3. Детальные требования к этапам 3–4 (Telegram, приложения, синхронизация) определяются отдельным техническим заданием «Платформы».

4 Требования к выполнению работ по разработке Системы

4.1 Общие принципы создания Системы

При создании Системы необходимо руководствоваться следующими принципами:

- Принцип достоверности («цифры важнее функций»): новая функция не выпускается, пока в отчётах есть известные ошибки; при отсутствии данных для расчёта (например, курса) Система показывает предупреждение, а не подставляет значение.
- Принцип двойной записи: каждая операция состоит из проводок, сумма которых по каждой валюте равна нулю; остатки счетов и итоги не хранятся, а вычисляются из проводок.
- Принцип изоляции проектов: все данные, кроме пользователей, принадлежат проекту; каждый запрос к данным ограничивается проектом и ролью пользователя, проверяемыми на сервере.
- Принцип модульности (гексагональная архитектура): одна функциональная область – один программный модуль (крейт) со своей моделью, миграциями, REST API, страницами и тестами; модули не обращаются друг к другу напрямую, а связываются через порты и адаптеры в модуле сборки приложения.
- Принцип «одно ядро – тонкие входы»: веб-интерфейс, REST API, MCP и будущие клиенты (Telegram, приложения) вызывают одни и те же сервисы модулей и одну проверку прав; сервер – один исполняемый файл.
- Принцип трёх языков: всё, что видит пользователь, сразу реализуется на русском, английском и тайском языках; данные хранятся в оригинале с переводами.
- Принцип обратимости действий: удаление и архивирование выполняются без вопроса «Вы уверены?» и отменяются кнопкой «Отменить»; через ассистента удаление невозможно.
- Принцип стандартизации (унификации): модули строятся по эталонной архитектуре rust-hex-api; справочники имеют единые поля (код, название с переводами, признак архива) и единое поведение (архив вместо удаления).
- Принцип открытых форматов: обмен данными с внешними клиентами – JSON по HTTPS; описание REST API – OpenAPI (Swagger).

4.2 Требования к Системе в целом

Основной целью разработки Системы является автоматизация учёта денежных средств семьи с достоверными остатками и бюджетом.

Целью развития Системы является создание единой среды для:

- ввода операций там, где они появляются: в веб-интерфейсе, через ИИ-ассистента, из SMS и уведомлений банка, в Telegram и в приложении на телефоне, в том числе без сети;
- совместного ведения бюджета семьёй с разграничением прав;
- анализа расходов по категориям, местам, людям и периодам.

4.2.1 Требования к структуре и функционированию Системы

4.2.1.1 Перечень компонентов Системы, их назначение и основные характеристики

Система должна находиться в режиме эксплуатации в составе следующих модулей (функциональных компонентов):

- «Проекты и доступ» (projects), где должны реализовываться:

  - пользователи, проекты, участники и роли, приглашения;
  - серверные сессии, личные API-токены, журнал безопасности, профиль пользователя;
  - вход через Rauthy, онбординг нового пользователя, гостевой демо-вход.
- «Валюты» (currencies) – справочник валют проекта.
- «Курсы валют» (rates) – курсы валют к базовой валюте по датам.
- «Счета» (accounts) – справочник счетов с остатками из журнала операций.
- «Категории» (categories) – дерево категорий расходов и доходов.
- «Места» (places) – справочник мест оплаты с категорией и счётом по умолчанию.
- «Люди» (people) – справочник людей для отметки «Для кого».
- «Операции» (transactions), где должны реализовываться:
  - операции по двойной записи: расход и доход с позициями чека, перевод и обмен валют, начальный остаток;
  - лента операций, выписка по счёту, сверка остатка, быстрый ввод;
  - приём операций от внешних клиентов без дублей (внешние записи).
- «Лимиты» (limits) – лимиты расходов на поддерево категорий по периодам.
- «Планирование» (planning) – плановые операции, календарь платежей, оплата и пропуск.
- «Бюджет» (budget) – экран «Обзор»: план, факт, прогноз и лимит по дереву категорий; собственных таблиц не имеет.
- «Ассистенты» (mcp) – MCP-сервер для подключения Claude и ChatGPT.
- «Сайт» (site) – лендинг, правовые страницы, sitemap.xml и robots.txt в приложении.
- Ядро (kernel) – общие механизмы: ошибки, аутентификация и авторизация, CSRF, лимиты запросов, конфигурация, криптография, многоязычность, каркас веб-интерфейса, «Отменить», статические ресурсы, страницы 404 и 500.
- Модуль сборки приложения (app) – запуск, связывание модулей через порты, демо-данные, фоновые задачи.
- Сервер входа Rauthy – регистрация, вход по паролю, через GitHub и Facebook, второй фактор, сброс пароля.
- СУБД PostgreSQL – хранение данных Системы (база family_budget) и сервера входа (база rauthy).
- Обратный прокси Caddy – HTTPS-сертификаты и терминирование TLS в продакшене.
- Сайт с блогом (Zola) – лендинг, «Что нового», гайды, идеи, правовые страницы на трёх языках; публикуется отдельно от приложения.

4.2.1.2 Требования к способам и средствам связи для информационного обмена между компонентами Системы

В качестве протокола взаимодействия между компонентами Системы на транспортно-сетевом уровне используется протокол TCP/IP.

Информационное взаимодействие между модулями приложения должно осуществляться внутри одного процесса через порты (интерфейсы) и адаптеры, связанные в модуле сборки приложения. Внешние ключи СУБД допускаются только внутри модуля; ссылки между модулями выполняются по коду записи справочника и проверяются сервисами через порты.

Информационный обмен между компонентами Системы должен осуществляться в следующих режимах:

- синхронно по запросу пользователя (веб-интерфейс, REST API, MCP);

- периодически по расписанию фоновых задач приложения (очистка журнала безопасности, сброс демо-данных);
- через единое хранилище данных (СУБД PostgreSQL).

Для информационного обмена должны использоваться следующие протоколы:

- HTTPS (TLS версии не ниже 1.2) – между браузером, ассистентами, внешними клиентами и обратным прокси;
- HTTP/1.1 – между обратным прокси и приложением, приложением и Rauthy внутри сети контейнеров;
- протокол PostgreSQL – между приложением, Rauthy и СУБД;
- OpenID Connect – между приложением и Rauthy;
- MCP (Streamable HTTP, JSON-RPC 2.0) – между ассистентами и приложением.

4.2.1.3 Требования к характеристикам взаимосвязей Системы с внешними и со смежными системами, обеспечению ее совместимости

Система обеспечивает информационные потоки, указанные в таблице ниже (Таблица 1).

Таблица 1 – Информационные потоки, обеспечиваемые Системой

№	Внешняя или смежная система	Описание передаваемых данных
1.	Сервер входа Rauthy	Регистрация, вход, второй фактор; передача в Систему подтверждённого пользователя (ID-токен: sub, email, имя, признак второго фактора amr); завершение сессии; удаление пользователя при удалении аккаунта
2.	GitHub, Facebook (через Rauthy)	Вход через внешних поставщиков; Система с ними напрямую не взаимодействует
3.	Почтовый сервер (SMTP)	Письма Rauthy: регистрация, сброс пароля, код второго фактора
4.	Claude (claude.ai, приложения Claude, Claude Desktop, Claude Code)	Контекст проекта, операции, остатки, бюджет, плановые платежи по MCP; создание и изменение данных в пределах прав ассистента
5.	ChatGPT	То же, что для Claude, с отдельным проектом и правами
6.	Внешние клиенты пользователя (синхронизация SMS банка, скрипты)	Создание операций и внешних записей через REST API по личному API-токену
7.	GitHub (репозиторий, Actions, GHCR)	Исходный код, автоматические проверки, сборка и публикация образа Системы
8.	GitHub Pages	Публикация сайта с блогом

Информационное взаимодействие с внешними системами должно осуществляться с помощью веб-сервисов (REST, MCP, OpenID Connect) и форматов данных на основе JSON.

В случае если взаимодействие с внешней системой не может быть проверено на продуктивной среде, функции взаимодействия должны быть проверены на стенде разработки (Rauthy в контейнере с демо-пользователями, вход для разработки) или с использованием поддельных (фейковых) реализаций портов в автоматических тестах.

4.2.1.4 Требования к режимам функционирования Системы

Система должна функционировать в следующих режимах:

- штатный режим, в котором все модули корректно выполняют все свои функции;
- сервисный режим, в котором выполняются обновление версии, восстановление из резервной копии и иные регламентные работы, при этом Система может быть кратковременно недоступна;
- аварийный режим, в котором один или несколько компонентов Системы (приложение, СУБД, Rauthy, обратный прокси) не выполняют своих функций.

Основным режимом функционирования является штатный режим. В штатном режиме Система должна обеспечивать круглосуточную работу пользователей и ассистентов.

Дополнительно приложение должно поддерживать режимы окружения:

- продакшен (APP_ENV=prod): вход только через Rauthy, обязательные секреты и HTTPS, Swagger и демо-данные выключены;
- разработка (APP_ENV=dev): допускается «Вход для разработки» под демо-пользователями при пустом адресе Rauthy, доступен Swagger.

В продакшене приложение не должно запускаться без секрета сессии, настроек клиента OpenID Connect и HTTPS.

Допускается временное ограничение доступности Системы:

- в периоды обновления версии и иных регламентных работ;
- при отказе сервера входа Rauthy – для новых входов (действующие сессии продолжают работать);
- при отказе внешних сервисов ассистентов – для работы через ассистентов.

4.2.1.5 Требования по диагностированию Системы

Для диагностирования Системы должны использоваться штатные средства контейнерной среды (журналы и состояние контейнеров), журналы СУБД и встроенные средства приложения.

Приложение должно предоставлять адрес проверки работоспособности (/health). На этапе 1 развития должны быть добавлены: адрес проверки готовности с запросом к СУБД, структурированные журналы приложения (JSON) с идентификатором запроса, базовые метрики, корректное завершение работы по сигналу SIGTERM без обрыва запросов на запись.

Диагностированию подлежат:

- сбои и нарушения функционирования приложения (ошибки 500 фиксируются в журнале приложения, пользователю показывается страница 500 на его языке);
- недоступность СУБД и сервера входа;
- некорректные ответы сервера входа (ошибки входа фиксируются в журнале, пользователю показывается общий текст без технических подробностей);
- сбои при выполнении резервного копирования и проверки восстановления.

4.2.2 Требования к локализации модулей Системы

Каждый модуль Системы должен быть реализован на трёх языках. Требования к локализации модулей приведены в таблице ниже (Таблица 2).

Таблица 2 – Локализация модулей Системы

Модуль	Строки интерфейса и ошибки	Переводимые данные	Не переводится
Проекты и доступ	ru, en, th	–	название проекта (вводится как есть)
Валюты	ru, en, th	название валюты	код, символ
Курсы валют	ru, en, th	–	–
Счета	ru, en, th	название счёта	код
Категории	ru, en, th	название категории	код
Места	ru, en, th	–	название места (как на вывеске), варианты написания
Люди	ru, en, th	–	имя
Операции	ru, en, th	описание операции	название и заметка позиции
Лимиты	ru, en, th	–	–
Планирование	ru, en, th	название плана	–
Бюджет («Обзор»)	ru, en, th	–	–
Ассистенты (MCP)	ru, en, th (ответы инструментов – на языке пользователя)	переводы передаются ассистентом	–
Сайт и лендинг	ru, en, th	–	–

4.3 Требования к функциям модулей Системы

Требования к функциям модулей приведены в порядке разделов меню Системы: основное меню («Обзор», «Операции», «Календарь», «Лимиты»), группа «Справочники» («Категории», «Счета», «Места», «Люди», «Валюты», «Курсы»), служебные разделы («Проекты и участники», «Профиль», «Ассистенты (MCP)») и публичная часть (лендинг и правовые страницы).

Для всех модулей действуют общие требования:

- все данные модуля принадлежат проекту; запрос к записи чужого проекта должен завершаться ответом «не найдено» (404) как в интерфейсе, так и в REST API;
- функции изменения данных доступны ролям OWNER и EDITOR; для роли VIEWER кнопки изменения не отображаются, а любой изменяющий запрос отклоняется сервером с кодом 403 и сообщением «роль VIEWER: только просмотр»;
- удаление и архивирование выполняются без запроса подтверждения: после действия показывается всплывающее сообщение на 5 секунд с кнопкой «Отменить», удалённая запись хранится для восстановления не менее 120 секунд;
- у каждого списка должны быть предусмотрены состояния «пусто» (объяснение и кнопка первого действия), «ошибка» и «данные»;
- каждая функция веб-интерфейса должна иметь соответствующий метод REST API под адресом /api/<модуль>; страница списка REST API – не более 200 записей (по умолчанию 100).

4.3.1 Требования к функциям модуля «Обзор» (бюджет)

В рамках создания Системы необходимо создать модуль «Обзор» (программный модуль budget).

Модуль «Обзор» должен быть предназначен для отображения бюджета проекта за выбранный период: сколько уже потрачено и получено, сколько ещё предстоит по планам и сколько осталось до лимита – по каждой ветке дерева категорий. «Обзор» является стартовой страницей пользователя после входа.

Модуль не должен иметь собственных таблиц: все показатели вычисляются при каждом запросе из проводок, плановых операций, лимитов и курсов других модулей.

В модуле должны быть реализованы следующие функции:

- Функция выбора периода и режима сводки;
- Функция отображения сводки периода;
- Функция отображения таблицы расходов по дереву категорий;
- Функция отображения таблицы доходов;
- Функция отображения предстоящих плановых платежей;
- Функция быстрого расхода из строки категории.

4.3.1.1 Требования к функции выбора периода и режима сводки

Страница «Обзор» должна открываться по адресу /budget с параметрами view (week, month, year), date, member, mode (fact, forecast), sort и dir. По умолчанию: период «Месяц», дата – сегодня, режим «Факт».

Пользователь должен иметь возможность:

- выбрать период: «Неделя» (понедельник – воскресенье), «Месяц», «Год»;
- перейти к предыдущему и следующему периоду стрелками «‹» и «›» и вернуться к текущему кнопкой «Сегодня»;
- переключить «Режим сводки»: «Факт» или «Прогноз»;
- выбрать участника в поле «Участник» (поле отображается, если в проекте больше одного участника; по умолчанию «Все участники»). Фильтр по участнику должен влиять только на факт; лимиты и планы остаются общими.

4.3.1.2 Требования к функции отображения сводки периода

В сводке должны отображаться следующие показатели в базовой валюте проекта:

- «Доходы» и «Расходы»: в режиме «Факт» – факт крупно, ожидаемое по планам («Ожидается») и «Прогноз» мелко; в режиме «Прогноз» – прогноз крупно, «Факт · План» мелко;
- «Итог периода» (факт или прогноз);
- «Курсовая разница» – сумма проводок системного счёта «Обмен валют» за период;
- «Остатки на конец периода (прогноз)» – по каждому активному счёту.

Доходами и расходами должны считаться только проводки на категории: переводы между счетами и начальные остатки их не меняют, комиссии входят в расходы, курсовая разница показывается отдельной строкой.

Суммы в других валютах должны пересчитываться в базовую валюту по курсу на дату операции (последний курс не позже этой даты). Если курса валюты нет, её суммы не должны входить в итоги; у категории должна отображаться пометка «нет курса» с исходной суммой.

4.3.1.3 Требования к функции отображения таблицы расходов

Таблица «Расходы» должна отображаться в виде дерева категорий расходов со следующими колонками:

- Категория;
- Лимит – лимит категории, пересчитанный на период отчёта;
- Факт – сумма проводок на категорию и всех её потомков за период (возвраты вычитаются);
- План – неоплаченные вхождения плановых операций категории и потомков в периоде, включая просроченные;
- Прогноз = Факт + План;
- Доступно = min(Лимит − Прогноз, Доступно родителя); если доступное ограничено лимитом предка, должна отображаться отметка «ограничено лимитом родителя»;
- «%» – полоса заполнения лимита с цветовой индикацией.

Цветовая индикация полосы: прогноз не менее 80 % лимита – янтарный цвет, более 100 % – красный, иначе – зелёный. При нулевом лимите любая трата считается превышением.

Если сумма лимитов подкатегорий больше лимита категории, у категории должно отображаться предупреждение «Сумма лимитов подкатегорий больше лимита категории».

Дерево по умолчанию должно быть свёрнуто до 2-го уровня; кнопка «Развернуть всё» раскрывает все уровни. Архивные категории без движения за период не отображаются.

Должна быть реализована сортировка по щелчку на заголовке колонки: первый щелчок – по названию по возрастанию, по суммам по убыванию; второй – в обратном порядке; третий – исходный порядок дерева. Сортируются только соседние узлы одного уровня, пустые значения располагаются в конце.

Лимит на период отчёта должен вычисляться как сумма лимитов по каждой неделе или месяцу, попадающим в отчёт, с учётом даты начала действия лимита: месячный лимит в годовом отчёте – 12 месяцев (а не пропорция дней), неделя на стыке месяцев учитывает лимиты обоих месяцев.

4.3.1.4 Требования к функции отображения таблицы доходов

Таблица «Доходы» должна отображаться в виде дерева категорий доходов со следующими колонками: Категория, План, Факт, Итого. Лимиты для доходов не применяются.

4.3.1.5 Требования к функции отображения предстоящих плановых платежей

Блок «Ещё предстоит по планам» должен содержать неоплаченные и непропущенные вхождения плановых операций периода; просроченные вхождения выделяются красным цветом. Щелчок по вхождению открывает его карточку в модуле «Календарь».

4.3.1.6 Требования к функции быстрого расхода из строки категории

В строке категории таблицы расходов должна отображаться кнопка «+», открывающая быстрый ввод расхода с уже выбранной категорией. Кнопка не отображается для роли VIEWER. После сохранения расхода показатели «Обзора» должны обновляться без перезагрузки страницы.

4.3.1.7 Требования к REST API модуля

Должен быть реализован метод GET /api/budget с параметрами view, date, member, возвращающий: границы периода, базовую валюту, строки дерева (код, родитель, название, тип, лимит, факт, план, прогноз, доступно, уровень индикации), курсовую разницу и перечень валют без курса.

4.3.1.8 Требования к развитию модуля

На этапах развития в модуле должны быть реализованы:

- этап 1: определение «сегодня» и границ текущего периода по часовому поясу пользователя (а не сервера); округление после суммирования по точности валюты; материализованная таблица месячных сумм по категориям – при превышении 100 тыс. проводок в проекте;
- этап 2: месяц, начинающийся с дня зарплаты; буддийский календарь для тайского языка;
- этап 5: графики расходов по месяцам, структура и тренды категорий, капитал в базовой валюте с переоценкой; отчёты по местам и по получателям («Для кого»); перенос остатка лимита и цели накоплений;
- этап 6: сводный отчёт по вложенным проектам.

4.3.2 Требования к функциям модуля «Операции»

В рамках создания Системы необходимо создать модуль «Операции» (программный модуль transactions).

Модуль «Операции» должен быть предназначен для ввода, просмотра, изменения и удаления операций по двойной записи: расходов и доходов с позициями чека, переводов и обмена валют между счетами, начальных остатков счетов, а также для формирования выписки и сверки остатка счёта.

В модуле должны быть реализованы следующие функции:

- Функция отображения ленты операций;
- Функция фильтрации ленты операций;
- Функция формирования выписки по счёту;
- Функция ввода и изменения операции (подробная карточка);
- Функция быстрого ввода;
- Функция сверки остатка счёта;
- Функция перевода описания операции;
- Функция удаления операции с отменой;
- Функция приёма операций от внешних клиентов (внешние записи).

4.3.2.1 Требования к модели операции и правилам двойной записи

Операция должна состоять из шапки и двух или более проводок. Шапка операции должна содержать:

- дату операции (occurred_on);
- тип операции: «Расход» (EXPENSE), «Доход» (INCOME), «Перевод» (TRANSFER), «Начальный остаток» (OPENING);
- описание на языке ввода, язык оригинала и переводы на русский, английский и тайский языки;
- автора операции (участник, внёсший операцию);
- ссылку на оплаченное вхождение плана (идентификатор плана и дата вхождения) – только у операций, созданных из плана;
- счёт, итоговую сумму, признак возврата и место – у расхода и дохода.

Проводка должна содержать: вид ссылки (счёт ACCOUNT, категория CATEGORY или системный счёт SYSTEM), код ссылки, валюту, сумму со знаком (не равную нулю), примечание и порядковый номер. Знак «+» означает, что счёт пополнился или деньги ушли на категорию расхода; знак «−» – что счёт уменьшился или деньги пришли из категории дохода. Знаки проводок пользователю не показываются.

Должны соблюдаться следующие правила:

- сумма проводок операции по каждой валюте равна нулю; правило проверяется сервисом модуля и отложенным триггером СУБД, поэтому несбалансированную операцию невозможно сохранить ни через API, ни прямой записью в БД;
- категории в проводках – только своего типа: в расходе и комиссии – категории расходов, в доходе – категории доходов; проводка на категорию – в валюте счёта;
- возврат – расход (доход) с обратными знаками, уменьшающий факт категории;
- изменение операции заменяет набор проводок целиком в одной транзакции СУБД; тип операции можно сменить;
- ссылки на счёт, категорию, место и человека должны вести на активные записи нужного типа; ссылка, уже использованная в операции, остаётся допустимой после архивирования записи;
- сумма проводки – по модулю меньше 10^{14}, не более 4 знаков после запятой и не более числа знаков валюты; количество и цена позиции – до 6 знаков после запятой.

Правила построения проводок по типам операций приведены в таблице ниже (Таблица 3).

Таблица 3 – Построение проводок по типам операций

Тип операции	Пример	Проводки
Расход	Чек 1 250 ฿	Карта −1 250; Продукты +900; Бытовая химия +250; Кафе +100
Доход	Зарплата	Карта +100 000; Зарплата −100 000
Перевод	Снятие наличных	Карта −5 000; Наличные +5 000
Перевод с обменом и комиссией	10 000 ₽ → 3 912,40 ฿, комиссия 50 ₽	Карта RUB −10 050 ₽; Комиссии +50 ₽; Обмен валют +10 000 ₽ и −3 912,40 ฿; Счёт THB +3 912,40 ฿
Начальный остаток	Из формы счёта	Счёт +12 000; Начальные остатки −12 000

4.3.2.2 Требования к функциям работы с лентой операций

4.3.2.2.1 Требования к функции отображения ленты операций

Лента операций должна открываться по адресу /transactions и отображать операции выбранного месяца в виде карточек, сгруппированных по дням (подпись дня – день недели и дата, например «Пт 09.10»). Порядок – по дате по убыванию, затем по времени создания по убыванию.

Карточка операции должна содержать:

- заголовок – описание операции, а при его отсутствии – место, «Перевод», «Начальный остаток» или категория;

- значки: «нет перевода» описания на язык пользователя (например, «RU?»), получатель, «SMS» (с подсказкой «Из SMS банка» или «Подтверждено SMS банка» и временем сообщения), «по плану»;
- строку сведений: место, счёт (у перевода – «откуда → куда»), автор;
- позиции чека, сгруппированные по категориям, в формате «Круассан · 2 шт × 25 − 3 → 47»;
- сводку получателей (например, «Для всех 88,00 ฿ · Для себя 45,00 ฿»), если получатели позиций различаются;
- пояснения: «Возврат», курс обмена («Курс 1 THB = 2,556 RUB»), комиссия с категорией;
- сумму со знаком и валютой; у перевода с обменом – вторую сумму в валюте зачисления.

Курс перевода не хранится, а показывается отношением сумм в направлении, где число больше 1; рядом должен отображаться курс по справочнику.

Над лентой должны отображаться итоги месяца: «Доходы», «Расходы», «Итог» по каждой валюте (базовая валюта первой), рассчитанные только по проводкам на категории и по всем операциям месяца.

Лента должна отображать не более 5 000 операций месяца; при превышении показывается предупреждение «Показаны последние 5 000 операций месяца – итоги посчитаны по всем».

У начального остатка в ленте кнопка «Удалить» не отображается: начальный остаток изменяется в форме счёта.

На странице должны отображаться кнопки «Расход» (клавиша N), «Доход» (I) и «Перевод» (T).

4.3.2.2.2 Требования к функции фильтрации ленты операций

Система должна обеспечивать фильтрацию ленты операций по следующим критериям:

- «Месяц» – переключение стрелками «‹» и «›», по умолчанию текущий месяц;
- «Все счета» / счёт;
- «Все типы» / «Расходы», «Доходы», «Переводы», «Начальные остатки»;
- «Все участники» / автор операции – поле отображается, если в проекте больше одного участника;
- «Все получатели» / «Для всех» / человек – поле отображается, если в справочнике «Люди» есть активные записи кроме текущего пользователя; итоги при этом считаются только по позициям выбранного получателя;
- флажок «Описание не на моём языке».

Фильтр должен применяться автоматически при изменении поля; без JavaScript – кнопкой «Показать».

REST API должен дополнительно поддерживать фильтры по периоду (from, to), месту, получателю и поставщику внешней записи.

4.3.2.2.3 Требования к функции формирования выписки по счёту

При выборе счёта лента должна превращаться в выписку с заголовком «<Счёт> · выписка»: после каждой операции отображается остаток счёта, а итоги заменяются показателями «Остаток на начало», «Поступило», «Списано», «Остаток на конец».

Нарастающий остаток должен отображаться, только если кроме счёта не задано других фильтров; при фильтре «Для кого» выписка не формируется.

Остаток счёта на дату должен вычисляться как сумма проводок счёта с датой не позже этой даты и совпадать с остатком в справочнике счетов и в «Обзоре».

4.3.2.3 Требования к функции ввода и изменения операции (подробная карточка)

Форма новой операции должна открываться по адресу /transactions/new (заголовок «Новая операция», при оплате плана – «Оплата по плану»), форма изменения – по адресу /transactions/{id}/edit (заголовок «Операция»). Поля формы должны предзаполняться из параметров адреса: тип, счёт, счёт зачисления, категория, сумма, дата, место, описание, план и дата вхождения, адрес возврата. Для начального остатка вместо формы операции должна открываться форма счёта.

Общие поля формы:

- «Тип» – «Расход», «Доход» или «Перевод», по умолчанию «Расход»;
- «Дата» – обязательное, по умолчанию сегодня;
- «Счёт» (у перевода – «Откуда») – обязательное;
- «Описание · необязательно» – до 300 символов, «Язык оригинала» (по умолчанию язык интерфейса), раскрывающийся блок «Переводы описания» с полями RU, EN, TH.

Поля расхода и дохода:

- «Место · необязательно» – отображается, если в справочнике есть места. Место подставляет счёт (если не указан) и категорию новых позиций (если тип категории совпадает); используются значения места или вышестоящей сети;
- блок «Ещё»: «Скидка на чек» (делится по позициям пропорционально сумме; остаток округления относится к самой крупной позиции), «Для кого по умолчанию» («Для всех», «Для себя» или человек), флажок «Разные получатели в чеке»;
- флажок «Возврат»;
- блок «Позиции» с итоговой суммой «Итого» и кнопкой «Добавить позицию».

Строка позиции должна содержать:

- «Категория» – выбор из дерева категорий типа операции;
- «Сумма»;
- «Название» – до 200 символов;
- переключатель «× цена», раскрывающий поля «Количество» (по умолчанию 1), «Единица» (шт, кг, л, г), «Цена», «Скидка»; сумма позиции при этом вычисляется как количество × цена, округлённое до точности валюты, минус скидка;
- «Для кого» – при включённом флажке «Разные получатели в чеке»;
- кнопку «Убрать».

При снятии флажка «Разные получатели в чеке» форма должна спрашивать «Сбросить всё на «…»?» с кнопками «Сбросить» и «Оставить разными». Получатель шапки операции не хранится и выводится из позиций. Позиция без получателя относится ко всем («Для всех»).

Поля перевода:

- «Куда» – счёт зачисления, отличный от счёта списания;
- «Списано» – всего ушло со счёта, вместе с комиссией;
- «Зачислено» – только при разных валютах счетов;
- строка курса с курсом «по справочнику» и отклонением;
- блок «Комиссия · необязательно»: «Категория комиссии» (только категории расходов) и «Сумма». Комиссия является расходом; курс считается без неё.

При изменении существующей операции форма должна показывать свёрнутый блок «Проводки» с пояснением «Двойная запись: сумма проводок по каждой валюте равна нулю» и сведения о связанных SMS банка.

Без JavaScript форма должна работать с тремя пустыми строками позиций в запасе.

До сохранения форма должна показывать, сколько останется доступно в категории с учётом лимитов предков, и предупреждать при достижении 80 % и превышении 100 % лимита; сохранить операцию можно всегда. Если для валюты счёта нет курса к базовой валюте, форма должна предупреждать: «Нет курса … к базовой валюте – сумма не войдёт в бюджет, пока не добавите курс».

4.3.2.3.1 Требования к проверкам формы операции

Система должна проверять введённые данные и показывать ошибку под соответствующим полем (у позиции чека – в строке этой позиции) на языке интерфейса. Перечень проверок приведен в таблице ниже (Таблица 4).

Таблица 4 – Проверки формы операции

Группа	Сообщение об ошибке
Позиции и суммы	«выберите категорию»; «введите сумму или цену»; «сумма должна быть больше нуля»; «у {валюты} не больше {n} знаков после запятой»; «количество должно быть больше нуля»; «единица – шт, кг, л или г»; «цена не может быть отрицательной»; «скидка не может быть отрицательной»; «скидка съедает сумму позиции целиком»; «скидка на чек должна быть меньше суммы позиций»; «сумма {сумма} не сходится с ценой: должно быть {расчёт}»; «нужна хотя бы одна позиция»; «не больше {n} позиций» (не более 100 позиций)
Перевод	«выберите счёт зачисления»; «введите сумму списания»; «счёт зачисления должен отличаться от счёта списания»; «комиссия должна быть меньше суммы списания»; «при одной валюте зачисляется списанное за вычетом комиссии»; «при разных валютах укажите, сколько зачислено»; «выберите категорию комиссии»; «введите сумму комиссии»
Справочники	«счёт '{код}' не найден»; «категория '{код}' не найдена»; «категория '{код}' – доходная, а нужна категория расхода» (и обратное); «место '{код}' не найдено или в архиве»; «человек '{код}' не найден или в архиве»
Оплата плана	«plan_id и plan_date задаются вместе»; «у плана нет платежа на {дату}»; «тип операции не совпадает с планом»; «платёж {дата} пропущен – сначала верните его в календаре»; «это вхождение плана уже оплачено другой операцией»
Начальный остаток	«начальный остаток не может быть нулевым»; «начальный остаток не оплачивает план»; «у счёта {код} уже есть начальный остаток – измените его в форме счёта»
Общие	«неверная дата»; «'{значение}' – не число»; «введите сумму»; «слишком большое число»

Введённые пользователем данные при ошибке не должны теряться. Сумма, присланная вместе с ценой и количеством, сверяется с расчётом с допуском в единицу младшего разряда валюты.

4.3.2.4 Требования к функции быстрого ввода

Быстрый ввод должен обеспечивать добавление расхода с любой страницы за три действия – сумма, категория, Enter – не дольше 5 секунд; остальные поля подставляются автоматически.

Точки входа: кнопка «Расход» в боковом меню, клавиши N (расход), I (доход), T (перевод), круглая кнопка «+» на нижней панели телефона, кнопка «+» в строке категории на «Обзоре», кнопка «Перевести» в карточке счёта. Горячие клавиши должны работать и в русской раскладке клавиатуры. Для роли VIEWER точки входа не отображаются.

Форма должна открываться поверх текущей страницы: на компьютере – диалогом, на телефоне – листом снизу. После сохранения пользователь остаётся на той же странице, а цифры страницы (бюджет, лента, остатки) обновляются без перезагрузки.

Поля быстрого ввода приведены в таблице ниже (Таблица 5).

Таблица 5 – Поля быстрого ввода

Поле	По умолчанию	Требования
Тип	Расход	Переключатель «Расход / Доход / Перевод»; заголовок «Новый расход», «Новый доход», «Новый перевод»
Сумма	пусто, фокус ввода	Цифровая клавиатура на телефоне; запятая и точка; простая арифметика «120+45» (подсказка «Можно считать: 120+45»)
Категория	не выбрана	До 6 чипов «Частые» – частые категории пользователя за 30 дней, дополненные корнями дерева; поиск «Найти категорию или место» на любом из трёх языков без учёта регистра и диакритики. Выбор места подставляет его категорию с пометкой «· из места» и кнопкой «Убрать место»
Счёт	последний счёт пользователя в расходах и доходах, иначе первый активный	Показывается валюта счёта
Дата	сегодня	Быстрые кнопки «Сегодня», «Вчера», «Другая…»
Куда, Зачислено	–	Только для перевода; «Зачислено» – при разных валютах
Описание	пусто	Необязательное; язык оригинала – язык интерфейса

Поведение быстрого ввода:

- кнопка «Подробная карточка» (у перевода – «С комиссией») открывает полную форму с уже введёнными значениями;
- Enter – «Сохранить», Ctrl/Cmd+Enter – «Сохранить и ещё» (счёт, дата и место сохраняются), Esc – закрыть;
- до сохранения форма показывает «Доступно» и «Останется доступно» в категории с учётом предков, «Больше 80 % лимита» и «Превышение лимита» с пояснением «Сохранить можно – это только предупреждение»;
- после сохранения показывается сообщение вида «Расход 250 ฿ · Кафе» с кнопкой «Отменить» на 5 секунд; «Отменить» удаляет операцию и показывает «Операция отменена»;

- при отсутствии счетов форма предлагает «Сначала добавьте счёт в «Справочники → Счета»».

Значения по умолчанию должны вычисляться методом GET /api/transactions/quick-defaults из проводок проекта без отдельных таблиц: счёт по умолчанию, частые категории, места (сначала частые), последний перевод, остатки лимитов. Сохранение выполняется обычным методом POST /api/transactions.

4.3.2.5 Требования к функции сверки остатка счёта

Форма «Сверить остаток» должна открываться по адресу /transactions/reconcile?account=<код> и содержать поля:

- «Дата» – по умолчанию сегодня;
- «Остаток по операциям» – вычисляется Системой, только для чтения;
- «Остаток на самом деле» – обязательное;
- «Категория» – категория расхода, если не хватает, или категория дохода, если остаток больше.

Разница должна вноситься операцией расхода или дохода с описанием «Сверка остатка». При совпадении остатков показывается сообщение «Остатки совпадают – вносить нечего», при успешной сверке – «Остаток сверен».

4.3.2.6 Требования к функции перевода описания операции

Кнопка «Перевести описание» должна открывать форму «Перевод описания» (/transactions/{id}/translate) с полями переводов на русский, английский и тайский языки; поле языка оригинала не заполняется. В ленте описание показывается на языке пользователя, а при отсутствии перевода – в оригинале с отметкой языка. В REST API должен быть реализован метод PUT /api/transactions/{id}/translation.

4.3.2.7 Требования к функции удаления операции с отменой

Удаление операции должно выполняться без подтверждения с сообщением «Удалено» и кнопкой «Отменить». Результат отмены – «Восстановлено» либо «Отменить уже нельзя – прошло слишком много времени».

Удаление операции, оплатившей вхождение плана, должно возвращать вхождение в статус «Ожидается».

4.3.2.8 Требования к функции приёма операций от внешних клиентов

Система должна принимать операции от внешних клиентов (программа синхронизации SMS банка, скрипты) через REST API по личному API-токену и связывать их с внешними записями.

Внешняя запись должна содержать: поставщика (латинские строчные буквы, цифры и «_», до 40 знаков), идентификатор записи (1–200 символов), время получения (RFC 3339), вид связи («CREATED» – операция создана по записи, «MATCHED» – запись подтвердила существующую операцию).

Должны соблюдаться следующие правила:

- ключ внешней записи – проект, поставщик и идентификатор; одна запись никогда не создаёт двух операций: повторный запрос с той же записью возвращает прежнюю операцию (код 200);
- если операция по записи была удалена, повторный запрос отклоняется (409, EXTERNAL_REF_DELETED); прикрепление записи к другой операции отклоняется (409, EXTERNAL_REF_TAKEN); начальный остаток со внешней записью не связывается;
- связь сохраняется после удаления операции;

- операции из SMS отмечаются в ленте значком «SMS»; ассистент дополняет такую операцию категорией и описанием вместо создания второй.

4.3.2.9 Требования к REST API модуля

В модуле должны быть реализованы следующие методы REST API:

- GET /api/transactions – список с фильтрами и постраничным выводом;
- POST /api/transactions – создание операции: клиент передаёт тип и поля формы, сервер строит проводки; ответ всегда содержит проводки;
- GET, PUT, DELETE /api/transactions/{id} – чтение, изменение, удаление;
- PUT /api/transactions/{id}/translation – переводы описания;
- GET /api/transactions/quick-defaults – значения по умолчанию быстрого ввода;
- GET /api/transactions/balances – остатки счетов на дату;
- POST /api/transactions/{id}/external-refs, GET /api/transactions/external-refs – внешние записи.

4.3.2.10 Требования к ролевой модели модуля

Просмотр ленты, выписки и карточек операций доступен всем ролям. Ввод, изменение, сверка, перевод описания и удаление операций доступны ролям OWNER и EDITOR. Для роли VIEWER кнопки изменения, быстрый ввод и горячие клавиши не отображаются, вверху страницы показывается отметка «Только просмотр».

4.3.2.11 Требования к развитию модуля

На этапах развития в модуле должны быть реализованы:

- этап 1: время операции и часовой пояс – операция хранит момент в UTC и свой пояс, время показывается в поясе операции одинаково для всех участников; отчёты строятся по дате операции;
- этап 1: приём без дублей – идентификатор клиента (client_id) с проверкой хеша тела запроса; поиск возможных дублей (тот же тип и признак возврата, счёт, сумма, место; разница времени не больше окна дублей проекта, по умолчанию 10 минут, поиск по дате ± 1 день); режимы on_duplicate: warn, skip, allow; пакетное создание операций с признаком атомарности;
- этап 1: мягкое удаление (отметка времени удаления и срок хранения) вместо хранения «Отменить» в памяти процесса; версия записи для защиты от одновременной правки;
- этап 1: курс из перевода – перевод с обменом, где одна из валют базовая, создаёт курс с источником «перевод»;
- этап 2: поиск операций по тексту, фильтры по категории, сумме и периоду, постраничный вывод ленты;
- этап 3: «Входящие» – разбор SMS, уведомлений, текста чеков и банковских выписок по шаблонам банков; импорт CSV и выписок с поиском дублей; экспорт в CSV; автопроведение плановых операций по желанию пользователя;
- этап 5: покупка в другой валюте (исходная сумма и сумма списания), кредитные карты, долги и займы, взаиморасчёты между участниками, теги, фото чека к операции.

4.3.3 Требования к функциям модуля «Календарь» (планирование)

В рамках создания Системы необходимо создать модуль «Календарь» (программный модуль planning).

Модуль «Календарь» должен быть предназначен для ведения плановых операций – ожидаемых расходов, доходов и переводов с правилом повторения – и для подтверждения их оплаты. Операция по плану не создаётся, пока пользователь не нажмёт «Оплатил» на конкретном вхождении. Разовая плановая операция – это план с правилом «Однократно».

В модуле должны быть реализованы следующие функции:

- Функция отображения календаря платежей;
- Функция отображения карточки платежа (вхождения);
- Функция оплаты и пропуска вхождения;
- Функция ведения реестра плановых операций;
- Функция расчёта вхождений по правилу повторения.

4.3.3.1 Требования к функции отображения календаря платежей

Календарь должен открываться по адресу /planning с параметром month (ГГГГ-ММ), по умолчанию – текущий месяц, и отображаться сеткой «Пн – Вс»; на телефоне – списком по дням. Навигация – стрелки «‹», «›» и кнопка «Сегодня».

Плашка вхождения должна содержать название плана, сумму со знаком «−» (расход), «+» (доход) или «⇄» (перевод) и, для валюты, отличной от базовой, сумму в базовой валюте («≈ …»). Цвет плашки определяется статусом вхождения.

Над календарём должны отображаться итоги месяца в базовой валюте по курсу на дату платежа:

- «Ещё к оплате» – открытые расходы месяца и под ним «Просрочено: N»;
- «Оплачено по планам»;
- «Ожидаемые доходы»;
- «Обозначения» – легенда статусов.

Переводы в суммы итогов не входят и учитываются только в счётчике просроченных. Если для валюты плана нет курса, должно показываться предупреждение «Нет курса к базовой валюте – эти суммы не вошли в итоги месяца» со ссылкой «Курсы валют».

4.3.3.2 Требования к статусной модели вхождения

Статус вхождения должен вычисляться на лету и не храниться в БД, кроме пропусков. Оплату хранит сама операция (идентификатор плана и дата вхождения); вхождение считается оплаченным, пока такая операция существует. Статусная модель вхождения приведена в таблице ниже (Таблица 6).

Таблица 6 – Статусная модель вхождения плановой операции

Статус	Отображение	Действие	Результат	Роль
Ожидается (EXPECTED)	нейтральный цвет	Оплатил	→ Оплачено (создаётся операция)	OWNER, EDITOR
		Пропустить	→ Пропущено	OWNER, EDITOR
Просрочено (OVERDUE): дата прошла, решения нет	красный цвет	Оплатил	→ Оплачено	OWNER, EDITOR
		Пропустить	→ Пропущено	OWNER, EDITOR
Оплачено (PAID)	зелёный цвет	Открыть операцию	Переход к операции	все роли
		Удаление операции	→ Ожидается / Просрочено	OWNER, EDITOR

Статус	Отображение	Действие	Результат	Роль
Пропущено (SKIPPED)	зачёркнуто	Вернуть в ожидание	→ Ожидается / Просрочено	OWNER, EDITOR

Пропуск оплаченного вхождения должен отклоняться (409, «вхождение уже оплачено – сначала удалите транзакцию»). Повторная оплата того же вхождения запрещена уникальным индексом СУБД по плану и дате вхождения.

4.3.3.3 Требования к функции отображения карточки платежа

Карточка платежа должна открываться по адресу /planning/occurrence/{план}/{дата} и содержать: название плана, строку «куда · счёт · повторение», значок статуса, поля «Дата по плану» и «Ожидаемая сумма», у оплаченного – «Оплачено» (дата) и «Сумма».

Кнопки карточки (для ролей OWNER и EDITOR):

- у открытого вхождения – «Оплатил» и «Пропустить» (с «Отменить»);
- у пропущенного – «Вернуть в ожидание»;
- у оплаченного – «Открыть операцию»;
- всегда – «Изменить» (переход к плану).

4.3.3.4 Требования к функции оплаты вхождения

Кнопка «Оплатил» должна открывать форму операции, заполненную из плана: дата оплаты (сегодня), счёт, категория или счёт зачисления, сумма, описание. Сумму, счёт и дату можно изменить. Сумма зачисления при обмене валют вводится при оплате.

При сохранении Система должна проверять, что у плана есть вхождение на эту дату, тип операции совпадает с планом, вхождение не пропущено и ещё не оплачено (тексты ошибок – Таблица 4).

4.3.3.5 Требования к функции ведения реестра плановых операций

4.3.3.5.1 Требования к функции отображения реестра плановых операций

Реестр «Плановые операции» должен открываться по адресу /planning/plans и отображаться в виде таблицы со следующими колонками:

- Название (с отметкой «остановлен» у неактивного плана);
- Категория (у перевода – «→ счёт зачисления»);
- Повторение (с множителем «× N»);
- Начало (— окончание);
- Сумма в валюте счёта.

Действия строки: «Остановить» / «Возобновить», «Изменить». Порядок – сначала активные, затем по названию.

4.3.3.5.2 Требования к форме плановой операции

Форма плана должна содержать следующие поля:

- «Название» – обязательное, с языком оригинала и переводами;
- «Тип» – «Расход» (по умолчанию), «Доход», «Перевод»;
- «Счёт» – обязательное;
- «Ожидаемая сумма» – обязательное, больше нуля, в валюте счёта;
- «Категория» – у расхода и дохода, того же типа;
- «Счёт зачисления» – только у перевода, отличный от счёта списания;
- «Повторение» – «Однократно», «Каждый день», «Каждую неделю», «Каждый месяц» (по умолчанию), «Каждый год»;
- «Каждые N» – целое от 1 до 366, по умолчанию 1;

- «Начало» – обязательное, по умолчанию сегодня;
- «Окончание» – необязательное, не раньше начала;
- «Активен» – по умолчанию включено.

Удаление плана выполняется с «Отменить»; при отмене план восстанавливается вместе с пропусками.

4.3.3.6 Требования к функции расчёта вхождений по правилу повторения

Вхождения должны вычисляться из правила на лету для видимого периода по правилам таблицы ниже (Таблица 7).

Таблица 7 – Правила повторения плановых операций

Правило	Пример	Расчёт дат
Однократно (ONCE)	Страховка 12.11	Только дата начала
Каждый день × N (DAILY)	Каждый день	Начало + k·N дней
Каждую неделю × N (WEEKLY)	Каждую 2-ю пятницу	Начало + k·N недель
Каждый месяц × N (MONTHLY)	Аренда 5-го числа	День месяца из даты начала; 31-е число в коротком месяце – последний день месяца
Каждый год × N (YEARLY)	Налог на авто 1 марта	День и месяц из даты начала; 29.02 в невисокосный год – 28.02

Должны соблюдаться ограничения: не более 3 700 вхождений за один расчёт, окно запроса вхождений – не более 1 098 дней. Остановленный план показывает только решённые (оплаченные и пропущенные) вхождения.

Изменение плана должно влиять только на будущие нерешённые вхождения. Если меняется правило дат, а по плану уже есть оплаты или пропуски, Система должна закрывать прежний план (окончание – вчера) и создавать продолжение серии с новым правилом от даты изменения, сохраняя прошлые оплаты; пользователю показывается соответствующее сообщение.

4.3.3.7 Требования к REST API модуля

Должны быть реализованы методы: /api/planning/plans (создание, список, чтение, изменение, удаление), POST и DELETE /api/planning/plans/{id}/skips/{дата} (пропуск и возврат), GET /api/planning/occurrences?from&to (вхождения за период).

4.3.3.8 Требования к развитию модуля

На этапах развития в модуле должны быть реализованы:

- этап 3: напоминания о плановых платежах в Telegram и по email; автопроведение плановых операций по выбору пользователя;
- этап 4: push-напоминания в приложениях для телефона.

4.3.4 Требования к функциям модуля «Лимиты»

В рамках создания Системы необходимо создать модуль «Лимиты» (программный модуль limits).

Модуль «Лимиты» должен быть предназначен для задания пределов расходов на категорию и всё её поддерево за неделю, месяц или год в базовой валюте проекта.

В модуле должны быть реализованы следующие функции:

- Функция отображения реестра лимитов;

- Функция задания лимита;
- Функция снятия лимита с даты;
- Функция удаления лимита с отменой.

4.3.4.1 Требования к функции отображения реестра лимитов

Страница «Лимиты расходов» должна открываться по адресу /limits и содержать форму задания лимита, таблицу лимитов и кнопку «Бюджет по лимитам» (переход на «Обзор»).

Таблица лимитов должна содержать следующие колонки:

- Категория (с отступом по уровню дерева и кодом);
- Период («Неделя», «Месяц», «Год»);
- Сумма (или отметка «лимит снят»);
- «Действует с» (с пояснением «действует с <начало периода>» и отметкой «заменён» у лимита, заменённого более поздним);
- Удалить.

Порядок строк – по дереву категорий, внутри категории – по дате начала действия по убыванию. Лимиты архивных категорий не отображаются.

4.3.4.2 Требования к функции задания лимита

Форма задания лимита должна содержать следующие поля:

- «Категория» – только активные категории расходов, деревом;
- «Период» – «Месяц» (по умолчанию), «Неделя», «Год»;
- «Сумма, <базовая валюта>» – обязательное, не меньше нуля;
- «Действует с» – по умолчанию сегодня.

Должны соблюдаться следующие правила:

- лимит действует на всё поддерево категории: лимит на «Еду» учитывает траты на «Продукты» и «Кафе»;
- лимит действует с начала своего периода (неделя – с понедельника) до периода следующего лимита этой категории;
- лимит 0 означает «тратить нельзя совсем»;
- для одной категории и даты начала допускается только один лимит (иначе – ошибка «запись с таким ключом уже существует»).

Ошибки: «лимит не может быть отрицательным (0 – тратить нельзя совсем)», «укажите сумму лимита или снимите лимит», «неверная дата».

4.3.4.3 Требования к функции снятия лимита с даты

Кнопка «Снять лимит с даты» должна создавать правило без суммы: начиная с указанной даты категория не ограничивается, прошлые периоды сохраняют лимит. В таблице такое правило отображается с отметкой «лимит снят».

4.3.4.4 Требования к расчёту показателей лимита

Для каждого узла дерева за выбранный период должны вычисляться показатели «Факт», «План», «Прогноз», «Остаток» (лимит − прогноз) и «Доступно» (минимум остатка узла и доступного у родителя) по правилам модуля «Обзор». Например, если у «Кафе» осталось 2 000 ฿, а у «Еды» – 500 ฿, потратить в кафе можно 500 ฿.

Предупреждения:

- прогноз не менее 80 % лимита – янтарный цвет, более 100 % – красный;
- сумма лимитов подкатегорий больше лимита родителя – предупреждение у родителя о несогласованных лимитах;
- при вводе позиции, выводящей узел или его предка за лимит, форма операции показывает предупреждение, но не запрещает сохранение.

4.3.4.5 Требования к REST API модуля

Должны быть реализованы методы /api/limits: создание, список, чтение, изменение, удаление; у снятого лимита в ответе передаётся признак removed: true.

4.3.4.6 Требования к развитию модуля

На этапе 3 должны быть реализованы напоминания о превышении лимита; на этапе 5 – перенос остатка лимита на следующий период (конверты) и цели накоплений.

4.3.5 Общие требования к справочникам

Справочники ведутся в каждом проекте отдельно. Ключ записи справочника – код: латинские буквы, цифры и «_», до 40 знаков, неизменяемый после создания; код уникален в пределах проекта.

Записи справочников не удаляются, а переносятся в архив: на архивную запись нельзя сослаться заново, прежние ссылки остаются действительными. Архивные записи отображаются в списках приглушёнными с отметкой «в архиве». Восстановление из архива выполняется кнопкой «Отменить» после архивирования.

Названия валют, счетов и категорий (а также планов и описаний операций) должны храниться по правилу трёх языков: оригинал на языке ввода, язык оригинала и переводы на русский, английский и тайский языки (колонка языка оригинала не заполняется). Показ на языке L: если L совпадает с языком оригинала – оригинал; иначе перевод на L, а при его отсутствии – оригинал с отметкой «нет перевода».

Форма переводимого названия должна содержать поле «Название» (обязательное, до 200 символов), «Язык оригинала» (RU, EN, TH, по умолчанию язык интерфейса) и раскрывающийся блок «Переводы» с тремя необязательными полями.

Общие сообщения об ошибках справочников: «не может быть пустым», «только латинские буквы, цифры и _, до 40 знаков», «запись '{код}' не найдена», «запись с таким ключом уже существует», «код '{код}' уже занят».

Для каждого справочника в REST API должны быть реализованы методы: POST (создание), GET (список, параметр include_archived), GET и PUT по коду, DELETE по коду (перенос в архив).

4.3.6 Требования к функциям модуля «Категории»

В рамках создания Системы необходимо создать модуль «Категории» (программный модуль categories).

Модуль «Категории» должен быть предназначен для ведения дерева статей расходов и доходов, к которым относятся позиции операций, лимиты и плановые операции.

В модуле должны быть реализованы следующие функции:

- Функция отображения дерева категорий;
- Функция создания и изменения категории;
- Функция архивирования и восстановления категории.

4.3.6.1 Требования к функции отображения дерева категорий

Страница «Категории» (/categories) должна содержать две карточки – «Расходы» и «Доходы» – с деревом категорий. Строка категории должна содержать: отступ по уровню, название, код, отметки «в архиве» и «нет перевода», действия «+ подкатегория» (только у активных) и «Изменить».

Порядок: корни – по типу, порядку сортировки и коду; потомки – по порядку сортировки и коду. Под деревом выводится подсказка о том, что лимиты и бюджет учитывают дерево.

4.3.6.2 Требования к форме категории

Форма категории (/categories/new с параметрами parent и kind, /categories/{код}/edit) должна содержать поля:

- «Код» – обязательное, до 40 символов (пример: FOOD_CAFE), при изменении только для чтения;
- «Тип» – «Расход» (по умолчанию) или «Доход»;
- «Название» с языком оригинала и переводами;
- «Родитель» – «— корень —» или активная категория того же типа, кроме самой категории;
- «Порядок» – целое число, по умолчанию 10.

4.3.6.3 Требования к правилам дерева категорий

Должны соблюдаться следующие правила:

- глубина дерева – не более 5 уровней («глубина дерева больше 5 уровней»);
- цикл запрещён: родителем нельзя назначить саму категорию или её потомка («не может быть родителем самой себя», «нельзя перенести категорию в собственного потомка»);
- родитель должен существовать и не находиться в архиве;
- тип потомка совпадает с типом родителя;
- тип категории нельзя изменить, если по ней есть операции, планы или лимиты;
- архивировать категорию можно, только если все её подкатегории в архиве; восстановить – только при активном родителе;
- проводки можно относить к любому узлу, а не только к листу.

При отказе в архивировании пользователь должен оставаться на странице категории, причина показывается во всплывающем сообщении на языке интерфейса, введённые правки названия не теряются.

4.3.6.4 Требования к категориям нового проекта

При создании проекта Система должна создавать набор категорий по умолчанию (21 категория расходов и доходов) с названиями на трёх языках.

4.3.7 Требования к функциям модуля «Счета»

В рамках создания Системы необходимо создать модуль «Счета» (программный модуль accounts).

Модуль «Счета» должен быть предназначен для ведения счетов проекта – наличных, карт, банковских и накопительных счетов – с остатками, вычисляемыми из журнала операций.

В модуле должны быть реализованы следующие функции:

- Функция отображения реестра счетов;
- Функция создания и изменения счёта с начальным остатком;
- Функция архивирования счёта.

4.3.7.1 Требования к функции отображения реестра счетов

Страница «Счета» (/accounts) должна отображать карточки счетов, включая архивные, в порядке кода. Карточка счёта должна содержать: название (с отметкой «нет перевода»), код, тип, «Остаток» на сегодня (отрицательный – красным цветом), «Держатель» (имя участника или «общий»), отметку «в архиве» и кнопки:

- «Выписка» – переход к ленте операций с фильтром по счёту;
- «Перевести» – быстрый ввод перевода с этого счёта;

- «Сверить» – форма сверки остатка.

Пустой список – «Счетов пока нет».

4.3.7.2 Требования к форме счёта

Форма счёта (/accounts/new – «Новый счёт», /accounts/{код}/edit – «Счёт») должна содержать поля:

- «Код» – обязательное, до 40 символов (пример: CARD_KBANK), при изменении только для чтения;
- «Название» с языком оригинала и переводами;
- «Тип счёта» – «Наличные», «Карта» (по умолчанию), «Банковский счёт», «Накопления»;
- «Валюта» – обязательное, по умолчанию базовая валюта проекта;
- «Остаток» (по умолчанию 0) и «на дату» (по умолчанию сегодня) – начальный остаток;
- «Держатель» – участник проекта или «общий»; пояснение: сведения только для справки, доступ не ограничивают.

Сохранение начального остатка должно создавать, изменять или (при нуле) удалять операцию «Начальный остаток» с проводками «Счёт / Начальные остатки». У счёта может быть не более одного начального остатка (частичный уникальный индекс СУБД), поэтому повторное нажатие «Сохранить» не записывает остаток дважды.

Ошибки: «по счёту есть операции – валюта остаётся {валюта}», «на счёте {код} остаток {сумма} – переведите его или сверьте остаток».

4.3.7.3 Требования к правилам счёта

Должны соблюдаться следующие правила:

- остаток на дату – сумма проводок счёта по эту дату, может быть отрицательным;
- валюту счёта нельзя изменить, если по счёту есть проводки;
- архивировать можно только счёт с нулевым остатком с учётом операций будущими датами;
- при создании проекта создаётся счёт «Наличные» (CASH) в базовой валюте, чтобы первую трату можно было записать без настройки.

4.3.7.4 Требования к развитию модуля

На этапе 2 коды счетов, категорий и валют должны генерироваться из названия; на этапе 5 должны поддерживаться кредитные карты (кредитный лимит, задолженность).

4.3.8 Требования к функциям модуля «Места»

В рамках создания Системы необходимо создать модуль «Места» (программный модуль places).

Модуль «Места» должен быть предназначен для ведения мест оплаты – магазинов, кафе, рынков, онлайн-сервисов – с категорией и счётом по умолчанию, которые подставляются в операцию.

В модуле должны быть реализованы следующие функции:

- Функция отображения реестра мест;
- Функция создания и изменения места;
- Функция архивирования места.

4.3.8.1 Требования к функции отображения реестра мест

Страница «Места» (/places) должна содержать вступительный текст и таблицу со следующими колонками:

- «Место» – название с отступом по уровню, адрес, варианты написания, отметка «в архиве»;
- «Тип»;
- «Подставляет» – «Категория · Счёт», при наследовании – с пометкой «· от сети»;
- «Код»;
- действия «магазин» (создать место внутри сети) и «Изменить».

Порядок – обход дерева, соседние места по названию без учёта регистра, затем по коду. Полное название места выводится в виде «Сеть › Магазин · Адрес» (например, «7-Eleven › Terminal 21 · Asok»).

4.3.8.2 Требования к форме места

Форма места должна содержать поля:

- «Название» – обязательное, до 200 символов, как на вывеске; не переводится;
- «Код»;
- «Входит в» – «— верхний уровень —» или активное место, кроме самого места, его потомков и мест нижнего уровня;
- «Филиал или адрес · необязательно»;
- «Тип» – «Магазин», «Кафе», «Рынок», «Онлайн», «Услуги», «Другое» (по умолчанию); новое место внутри сети берёт её название и тип;
- «Варианты написания» – через запятую, не более 20, без повторов (в том числе на других языках);
- «Категория по умолчанию» – только категории расходов; пустое значение – «— нет —» или «— как у «Сеть» —»;
- «Счёт по умолчанию».

4.3.8.3 Требования к правилам мест

Должны соблюдаться следующие правила:

- дерево мест – не более 3 уровней, без циклов; родитель должен быть активным;
- место без собственных категории или счёта берёт их у вышестоящей сети; поиск места выполняется и по вариантам написания сети;
- сеть с активными магазинами не архивируется («В «…» есть активные магазины – сначала архивируйте или перенесите их»); место не восстанавливается, пока его родитель в архиве;
- место указывается только у расхода и дохода;
- ассистент не создаёт место по чеку самостоятельно, а только с согласия пользователя и при наличии права «Места: добавлять и изменять».

Должен быть реализован метод REST API POST /api/places/{код}/restore (восстановление); в ответах передаются уровень, полное название, действующие категория и счёт.

4.3.9 Требования к функциям модуля «Люди»

В рамках создания Системы необходимо создать модуль «Люди» (программный модуль people).

Модуль «Люди» должен быть предназначен для ведения списка людей, для которых совершаются покупки («Для кого»): участников проекта и лиц без аккаунта (ребёнок, няня).

В модуле должны быть реализованы следующие функции:

- Функция отображения реестра людей;
- Функция создания и изменения записи человека;
- Функция синхронизации записей с участниками проекта.

4.3.9.1 Требования к функции отображения реестра людей

Страница «Люди» (/people) должна содержать таблицу со следующими колонками:

- «Человек» – имя (или «Удалённый участник»), отметки «это вы», «в архиве», другие имена;
- «Аккаунт» – «Участник проекта», «Аккаунт удалён» или «Без аккаунта»;
- «Код»;
- «Изменить».

Порядок – по имени без учёта регистра, затем по коду.

4.3.9.2 Требования к форме человека

Форма («Новый человек» – /people/new, «Человек» – /people/{код}/edit) должна содержать поля: «Имя» (обязательное, до 200 символов), «Код» (код ALL зарезервирован), «Другие имена · необязательно» (не более 20). Кнопка «В архив» доступна только для человека без аккаунта.

4.3.9.3 Требования к функции синхронизации с участниками

Должны соблюдаться следующие правила:

- запись участника создаётся автоматически при вступлении в проект; код формируется латиницей из имени (иначе MEMBER), при занятости – с суффиксом «_2»;
- выход из проекта переносит запись в архив, повторное вступление восстанавливает её;
- удаление аккаунта стирает имя, убирает связь с пользователем, отмечает запись «Удалённый участник» и переносит в архив; история покупок сохраняется;
- запись участника вручную не архивируется;
- в выборе «Для кого» собственная запись пользователя показывается как «Для себя»;
- ассистент добавляет человека только с согласия пользователя и при наличии права «Люди: добавлять и изменять».

4.3.9.4 Требования к развитию модуля

На этапах развития должна быть реализована связь записи человека без аккаунта с участником, когда этот человек получил аккаунт и вступил в проект (идея «Человек получил аккаунт»).

4.3.10 Требования к функциям модуля «Валюты»

В рамках создания Системы необходимо создать модуль «Валюты» (программный модуль currencies).

Модуль «Валюты» должен быть предназначен для ведения справочника валют проекта.

В модуле должны быть реализованы следующие функции:

- Функция отображения реестра валют;
- Функция создания и изменения валюты;
- Функция архивирования валюты.

4.3.10.1 Требования к функции отображения реестра валют

Страница «Валюты» (/currencies) должна содержать таблицу со следующими колонками: Код (с отметкой «базовая» у базовой валюты), Название (с отметкой «в архиве»), Символ, «Знаков после запятой», кнопка «Изменить». Порядок – по коду. Под таблицей – ссылка «Курсы к базовой валюте →». Пустой список – «Валют пока нет».

4.3.10.2 Требования к форме валюты

Форма валюты должна содержать поля:

- «Код» – обязательное, код ISO 4217 (подсказка «ISO 4217: THB, RUB, USD»), при изменении только для чтения;
- «Символ» – обязательное, до 5 символов;
- «Знаков после запятой» – целое от 0 до 4, по умолчанию 2;
- «Название» с языком оригинала и переводами.

Должны соблюдаться следующие правила:

- число знаков после запятой нельзя уменьшить, если в валюте уже есть суммы с большим числом знаков;
- базовая валюта проекта задаётся при создании проекта (3 латинские буквы), не меняется и не архивируется;
- валюту нельзя архивировать, если есть активные счета в этой валюте («есть активные счета (…) – сначала архивируйте их»).

4.3.10.3 Требования к развитию модуля

На этапе 1 при создании валюты число знаков после запятой должно подставляться по ISO 4217 (THB, RUB, USD – 2; VND, JPY – 0; KWD – 3; нестандартный код – 2); на этапе 2 мастер первого запуска должен предлагать валюты семьи с курсами.

4.3.11 Требования к функциям модуля «Курсы валют»

В рамках создания Системы необходимо создать модуль «Курсы валют» (программный модуль rates).

Модуль «Курсы валют» должен быть предназначен для ведения курсов валют проекта к базовой валюте по датам, используемых для пересчёта операций в бюджете, лимитах и календаре.

В модуле должны быть реализованы следующие функции:

- Функция отображения реестра курсов;
- Функция добавления курса;
- Функция удаления курса с отменой.

4.3.11.1 Требования к функции добавления курса

Форма добавления курса на странице «Курсы валют» (/rates) должна содержать поля:

- «Дата» – по умолчанию сегодня;
- «Валюта» – активные валюты, кроме базовой;
- курс «1 X = [ ] Y» с кнопкой «⇄ Поменять направление»: курс можно ввести в любую сторону («1 USD = 36,5 ฿» или «1 ฿ = 2,5 ₽»); направление по умолчанию – как у последнего курса этой валюты;
- подсказку с последним курсом и предпросмотр.

Если введённый курс отличается от предыдущего более чем в 5 раз, форма должна предупреждать «В {k} раз меньше/больше прошлого курса… Может, перепутано направление?».

Должны соблюдаться следующие правила:

- хранимый курс – число единиц базовой валюты за 1 единицу валюты; введённое значение и признак обратного ввода сохраняются;
- курс больше нуля, не более 6 знаков после запятой; для одной валюты и даты – один курс;
- пересчёт операции выполняется по последнему курсу не позже даты операции; для дат раньше первого курса используется самый ранний курс; если у валюты нет ни одного курса, суммы в итоги не входят и показывается предупреждение.

Ошибки: «введите курс», «'…' – не число», «курс должен быть больше нуля», «не больше 6 знаков после запятой», «слишком большой курс».

4.3.11.2 Требования к функции отображения реестра курсов

Таблица курсов должна содержать колонки: Дата, Курс (две строки – курс, как его ввели, и обратный), Удалить. Порядок – по дате по убыванию. Изменение курса в интерфейсе выполняется удалением и повторным вводом; удаление – с «Отменить». Пустая таблица – «Курсов пока нет».

4.3.11.3 Требования к развитию модуля

На этапах развития в модуле должны быть реализованы:

- этап 1: курс с моментом действия и источником («справочник», «перевод»); курс из перевода с обменом, где одна из валют базовая; ручной курс приоритетнее; пересчёт по одной функции для всех отчётов;
- этап 2: автоматическая загрузка курсов на дату из открытых источников (ЦБ РФ, Банк Таиланда или открытый API) с отметкой источника.

4.3.12 Требования к функциям модуля «Проекты и участники»

В рамках создания Системы необходимо создать модуль «Проекты и доступ» (программный модуль projects).

Модуль должен быть предназначен для управления пользователями, проектами (бюджетами), участниками и их ролями, а также входом в Систему, сессиями, личными API-токенами и журналом безопасности. Все данные Системы, кроме пользователей, принадлежат проекту; пользователь видит только проекты, в которых состоит.

В модуле должны быть реализованы следующие функции:

- Функция входа и регистрации через сервер входа;
- Функция онбординга нового пользователя;
- Функция гостевого демо-входа;
- Функция ведения проектов и переключения активного проекта;
- Функция управления участниками и ролями;
- Функция приглашения участников по ссылке;
- Функция требования второго фактора от участников;
- Функция удаления проекта и выхода из проекта.

4.3.12.1 Требования к функции входа и регистрации

Вход, регистрация, сброс пароля, вход через GitHub и Facebook, второй фактор (6-значный код по email, passkey) должны выполняться сервером входа Rauthy. Система не должна хранить пароли и обрабатывать вход сама: она получает от Rauthy подтверждённого пользователя и создаёт свою сессию.

Порядок входа:

- «Войти» или «Создать бюджет» → GET /auth/login → переход в Rauthy по схеме authorization code с PKCE S256; параметры state, nonce и верификатор PKCE хранятся в подписанной cookie на 10 минут; язык страниц Rauthy передаётся параметром ui_locales;
- Rauthy возвращает пользователя на /auth/callback; Система обменивает код на токены и проверяет ID-токен: подпись по ключам Rauthy (JWKS), издателя (iss), получателя (aud), nonce, срок действия; state сравнивается за постоянное время;
- пользователь ищется по идентификатору sub; email и имя обновляются из токена при каждом входе; аккаунт Facebook с тем же email считается другим пользователем, пока владелец не привяжет его в Rauthy;
- создаётся сессия Системы, токены Rauthy не сохраняются;
- «Выйти» (/auth/logout) завершает сессию Системы и сессию Rauthy.

При ошибке входа пользователю должен показываться общий текст без технических подробностей; подробности фиксируются в журнале безопасности (событие login_failed). При недоступности Rauthy Система отвечает кодом 503. Перенаправление после входа допускается только на адреса Системы.

4.3.12.2 Требования к функции онбординга

При первом входе пользователь должен пройти онбординг (/onboarding): имя, базовая валюта, язык. По завершении Система создаёт личный проект (PERSONAL) с базовой валютой, набором категорий по умолчанию и счётом «Наличные».

4.3.12.3 Требования к функции гостевого демо-входа

Кнопка «Посмотреть демо» на лендинге должна выполнять вход гостем без регистрации в демо-проект «Семья» с ролью VIEWER на 24 часа (POST /demo с обязательной проверкой заголовка Origin или Referer). Демо-проект пересоздаётся каждые 24 часа. Гостю запрещены профиль, токены, сессии, проекты, приглашения, удаление аккаунта и доступ ассистентов; язык гость меняет только для своего браузера. Демо-вход отключается параметром GUEST_DEMO.

4.3.12.4 Требования к функции ведения проектов

Страница «Проекты и участники» (/projects) должна содержать:

- карточки проектов пользователя: название, тип («Личный» / «Семейный»), базовая валюта, роль пользователя, отметка «2FA», кнопка «Переключиться» или отметка «Текущий»;
- для текущего проекта – участников, приглашения и настройки проекта;
- форму нового проекта: «Название» (до 100 символов), «Тип», «Базовая валюта».

Должны соблюдаться следующие правила:

- в личном проекте ровно один участник; кнопка «Сделать семейным» (только OWNER) переводит его в семейный;
- создатель проекта становится OWNER; в проекте всегда остаётся хотя бы один OWNER;
- активный проект хранится в cookie (веб) или задаётся API-токеном; переключатель проекта расположен вверху бокового меню;
- создание и удаление проекта выполняются атомарно (этап 1 развития).

4.3.12.5 Требования к функции управления участниками и ролями

В блоке участников должны отображаться: имя, email, выбор роли (OWNER, EDITOR, VIEWER), кнопка «Удалить». Изменять роли и удалять участников может только OWNER; изменение роли допускается только для существующих участников проекта. При удалении участника его API-токены этого проекта отзываются, а его запись в справочнике «Люди» переносится в архив.

4.3.12.6 Требования к функции приглашения участников по ссылке

Добавление человека в проект должно выполняться только ссылкой-приглашением: вступает лишь тот, кто сам открыл ссылку. Правила:

- приглашение создаёт только OWNER и только в семейном проекте; роль по приглашению – EDITOR или VIEWER (OWNER по приглашению не назначается);
- срок действия – 7 дней; одно приглашение на одного человека; не более 20 действующих приглашений на проект; истёкшие приглашения удаляются через 30 дней;
- код приглашения – случайный идентификатор и подпись HMAC-SHA256; сам код в БД не хранится;
- ссылку можно передать QR-кодом, через системное меню «Поделиться», Telegram и WhatsApp;
- GET /join/{код} только показывает приглашение, POST /join/{код} принимает его атомарно; если пользователь не вошёл, приглашение запоминается на сутки и принимается после входа;
- OWNER может отозвать приглашение.

4.3.12.7 Требования к функции требования второго фактора

OWNER семейного проекта должен иметь возможность включить «Требовать 2FA от участников». Признак второго фактора определяется по утверждению amr ID-токена (mfa, otp, hwk, swk, sms, fpt, face, iris, pop). Участник, вошедший без второго фактора, вместо данных проекта видит страницу с просьбой включить 2FA (код 403), а REST API и MCP отвечают кодом 403; страницы проектов, профиля, входа и онбординга остаются доступны. API-токен, выпущенный без второго фактора, перестаёт действовать в таком проекте.

4.3.12.8 Требования к функции удаления проекта и выхода из проекта

Удаление семейного проекта доступно только OWNER и подтверждается вводом названия проекта; удаление удаляет все данные проекта во всех модулях. Выход из проекта («Выйти из проекта») доступен участнику, если в проекте остаётся другой OWNER. Удаление проекта и выход из проекта – единственные действия, требующие подтверждения.

4.3.13 Требования к функциям модуля «Профиль»

В рамках создания Системы необходимо создать раздел «Профиль» пользователя в модуле «Проекты и доступ».

В разделе должны быть реализованы следующие функции:

- Функция изменения настроек пользователя;
- Функция управления устройствами (сессиями);
- Функция управления личными API-токенами;
- Функция просмотра журнала безопасности;
- Функция удаления аккаунта.

4.3.13.1 Требования к функции изменения настроек пользователя

Страница «Профиль» (/profile) должна содержать поля: «Имя» (до 100 символов), «Email» (только чтение, обновляется из Rauthy), «Язык» (RU, EN, TH), «Тема» («Системная», «Светлая», «Тёмная»). Ссылка ведёт в личный кабинет Rauthy для управления паролем, вторым фактором и привязками GitHub и Facebook.

4.3.13.2 Требования к функции управления устройствами

На странице «Безопасность» (/profile/security) должен отображаться список сессий пользователя: браузер и ОС, отметки «это устройство» и «с 2FA», время входа, последняя активность, IP-адрес, кнопка «Отозвать»; кнопка «Выйти везде» завершает все сессии.

Требования к сессиям:

- идентификатор сессии – 256 случайных бит в cookie «__Host-session» (HttpOnly, Secure, SameSite=Lax); в БД хранится только хеш SHA-256;
- неактивность – не более 7 дней, срок жизни – не более 30 дней; новый идентификатор при каждом входе; время последней активности обновляется не чаще раза в 5 минут.

4.3.13.3 Требования к функции управления личными API-токенами

Форма выпуска токена должна содержать поля: «Название», «Проект», «Права» («Только добавлять операции» – transactions:write; «Чтение» – read; «Чтение и запись» – write), «Срок» (30, 90 – по умолчанию, 365 дней или бессрочно).

Требования к токенам:

- формат – «fbt_» и 256 случайных бит; токен показывается один раз, в БД хранится только хеш SHA-256;
- токен действует в одном проекте и только для адресов /api и /mcp; запрос списка проектов возвращает только проект токена;
- права токена не превышают роль пользователя в проекте;
- токен запоминает, был ли второй фактор при выпуске; в проекте с требованием 2FA токен без второго фактора не выпускается и не действует.

Таблица токенов должна содержать колонки: Название, Проект, Права, Срок, Последнее использование, «Отозвать»; состояния токена – действует, отозван, истёк.

4.3.13.4 Требования к функции просмотра журнала безопасности

Пользователь должен видеть свои последние 100 событий журнала безопасности в таблице с колонками: Дата, Событие, Проект, IP. Состав событий приведен в разделе 8.5.

4.3.13.5 Требования к функции удаления аккаунта

Удаление аккаунта должно подтверждаться вводом email пользователя и удалять личные данные в Системе и пользователя в Rauthy (при наличии ключа администратора Rauthy). Удаление запрещено, если пользователь – единственный OWNER семейного проекта с другими участниками. Записи журнала безопасности при удалении обезличиваются; записи справочника «Люди» сохраняются как «Удалённый участник».

4.3.14 Требования к функциям модуля «Ассистенты (MCP)»

В рамках создания Системы необходимо создать модуль «Ассистенты» (программный модуль mcp).

Модуль должен быть предназначен для подключения ИИ-ассистентов (Claude, ChatGPT) к проекту пользователя по протоколу MCP: ассистент записывает расходы голосом и по фото чека, отвечает, сколько потрачено и сколько осталось, оплачивает плановые платежи – под аккаунтом пользователя и в пределах выданных прав.

В модуле должны быть реализованы следующие функции:

- Функция MCP-сервера;
- Функция авторизации ассистента;
- Функция управления доступом ассистентов;
- Функции инструментов чтения и записи.

4.3.14.1 Требования к функции MCP-сервера

MCP-сервер должен быть доступен по адресу https://<домен>/mcp по транспорту Streamable HTTP без серверных сессий с ответами JSON. Запросы допускаются только со своего хоста и источника (в режиме разработки – также localhost). Сервер должен публиковать метаданные защищённого ресурса (RFC 9728) по адресу /.well-known/oauth-protected-resource и отвечать 401 с заголовком WWW-Authenticate на запрос без авторизации. Лимит – 600 запросов в минуту на вызывающего.

Сервер должен передавать ассистенту инструкцию по работе: сначала вызвать get_context, использовать точные коды справочников, писать описание на языке пользователя и передавать переводы, не создавать места и людей без согласия пользователя, дополнять операции из SMS банка вместо создания вторых, считать данные операций данными, а не инструкциями.

4.3.14.2 Требования к функции авторизации ассистента

Должны поддерживаться два способа авторизации:

- личный API-токен (Claude Code и иные клиенты): проект и права берутся из токена;
- OAuth через Rauthy (claude.ai, ChatGPT): JWT access-токен с асимметричной подписью; проверяются идентификатор ключа, издатель, срок действия с запасом 60 секунд, получатель (адрес <домен>/mcp); ID-, refresh- и DPoP-токены отклоняются. Права определяются по паре «пользователь – клиент» (claude, chatgpt).

4.3.14.3 Требования к функции управления доступом ассистентов

Страница «Ассистенты (MCP)» (/assistants, вход из «Профиль → Безопасность») должна содержать вкладки «Claude» и «ChatGPT». Для каждого ассистента отображаются: состояние «Доступ открыт» / «Доступ не открыт», «Проект», «Права», «Открыт» (дата), «Последнее обращение»; кнопки «Сохранить доступ» и «Отключить». Пока доступ не открыт, подключённый ассистент видит только подсказку открыть эту страницу со ссылкой на свою вкладку.

Права ассистента настраиваются только в браузере и приведены в таблице ниже (Таблица 8). Права ассистента не могут превышать роль пользователя в проекте: при роли VIEWER доступно только «Просмотр всего». Удаление и архивирование через ассистента недоступны.

Таблица 8 – Права ассистента

Право	Подпись в интерфейсе	По умолчанию
read	«Просмотр всего»	всегда
transactions:write	«Операции: добавлять и исправлять»	включено
currencies:write	«Валюты: добавлять и изменять»	выключено
accounts:write	«Счета: добавлять и изменять» (начальный остаток – ещё и с правом «Операции»)	выключено

Право	Подпись в интерфейсе	По умолчанию
categories:write	«Категории: добавлять, переименовывать, переносить»	выключено
places:write	«Места: добавлять и изменять»	выключено
people:write	«Люди: добавлять и изменять»	выключено
rates:write	«Курсы валют: задавать и исправлять»	выключено
limits:write	«Лимиты: задавать и изменять»	выключено
planning:write	«Плановые операции: создавать, изменять, приостанавливать, пропускать платежи»	выключено

Флажок «Все справочники» должен выдавать все права справочников одним действием.

4.3.14.4 Требования к инструментам ассистента

Ассистенту должны быть доступны инструменты, приведенные в таблице ниже (Таблица 9). Список инструментов (tools/list) должен содержать только разрешённые инструменты; каждый вызов повторно проверяет права; каждое изменение фиксируется в журнале безопасности (событие mcp_write). Инструменты изменения записей помечаются признаком destructiveHint.

Таблица 9 – Инструменты MCP-сервера

Инструмент	Назначение	Право
get_context	Проект, базовая валюта, счета с остатками, дерево категорий на трёх языках, частые категории и места, люди, права ассистента	read
list_transactions	Операции за период с фильтрами (в том числе from_sms), до 200 на страницу	read
get_balances	Остатки счетов на дату	read
get_budget	Бюджет недели, месяца или года	read
list_upcoming_payments	Предстоящие и просроченные плановые платежи	read
list_plans, list_places, list_people, list_rates, list_limits	Просмотр планов, мест, людей, курсов, лимитов	read
add_expense, add_income	Расход или доход с позициями чека, местом, «Для кого», описанием и переводами	transactions:write
add_transfer	Перевод и обмен валют с комиссией	transactions:write
update_transaction	Изменение операции и отдельных позиций, дополнение операции из SMS	transactions:write
pay_planned	Оплата вхождения плана	transactions:write
skip_planned, add_plan, update_plan	Пропуск платежа, создание и изменение плана	planning:write
add_currency, update_currency	Валюты	currencies:write
add_account, update_account	Счета	accounts:write

Инструмент	Назначение	Право
add_category, update_category	Категории	categories:write
add_place, update_place	Места (только с согласия пользователя)	places:write
add_person, update_person	Люди (только с согласия пользователя)	people:write
set_rate	Курс валюты (в любом направлении)	rates:write
set_limit	Лимит категории	limits:write

Перед добавлением расхода или дохода, который мог уже прийти из SMS банка, ассистент должен искать операции из SMS за дату покупки ± 1 день и при совпадении суммы и направления дополнять найденную операцию. Отсутствие инструментов удаления должно проверяться автоматическим тестом.

4.3.14.5 Требования к развитию модуля

На этапах развития в модуле должны быть реализованы:

- этап 1: единая модель прав клиентов (права = роль ∩ права клиента), проверка прав в сервисах модулей вместо дублирования логики в MCP; страница «Подключённые приложения»;
- этап 3: инструмент черновика операции для «Входящих», импорт выписки и сценарий «итоги месяца», держатель счёта;
- этап 4: локальный MCP-сервер приложения для компьютера (stdio) для Claude Desktop;
- этап 6: Gemini и другие ассистенты – при подтверждении спроса.

4.3.15 Требования к функциям модуля «Сайт» (лендинг и правовые страницы)

В рамках создания Системы необходимо создать модуль «Сайт» (программный модуль site) без собственных таблиц.

Модуль должен быть предназначен для показа гостю продающей страницы (лендинга) и правовых страниц. Цель лендинга – регистрация; главная кнопка «Создать бюджет» расположена на первом экране и в конце страницы.

В модуле должны быть реализованы следующие функции:

- Функция отображения лендинга;
- Функция отображения правовых страниц;
- Функция поисковой оптимизации.

4.3.15.1 Требования к функции отображения лендинга

Адрес / для гостя должен показывать лендинг на языке браузера (ru, en, th; переключение – параметром ?lang=), для вошедшего – перенаправлять на «Обзор». Блоки лендинга сверху вниз:

- шапка: логотип, переключатель «RU · EN · TH», «Войти», «Создать бюджет»; при прокрутке закреплена;
- первый экран: заголовок-обещание, подзаголовок, кнопки «Создать бюджет» и «Войти», скриншот «Обзора»;
- выгоды – карточки с иконкой и двумя строками;
- секция о работе с ассистентом: сценарии, пример переписки, границы доступа;
- «Как это работает» – 3 шага: создайте бюджет → пригласите семью → добавляйте расходы;
- «Безопасность и приватность»;

- «Посмотреть демо» – гостевой вход в демо-проект;
- FAQ из 7 вопросов;
- финальный призыв и подвал со ссылками на /privacy, /terms, /data-deletion и контактами.

Слово «бесплатно» и блок тарифов не используются, пока не принято решение о монетизации. Сторонние трекеры не используются; воронка (визит → регистрация → первый расход) считается на сервере счётчиками без IP-адресов и cookie.

4.3.15.2 Требования к функции отображения правовых страниц

Должны быть реализованы страницы «Политика конфиденциальности» (/privacy), «Условия использования» (/terms) и «Удаление данных» (/data-deletion) на трёх языках. Страницы политики и удаления данных обязательны для работы входа через Facebook.

4.3.15.3 Требования к функции поисковой оптимизации

Должны быть реализованы: title, description и Open Graph на трёх языках, hreflang и canonical, /sitemap.xml (4 страницы × 3 языка с hreflang и x-default), /robots.txt с запретом индексации страниц приложения, /manifest.webmanifest. Страницы приложения должны отдаваться с заголовком X-Robots-Tag: noindex, nofollow.

4.3.15.4 Требования к сайту с блогом

Сайт с блогом (каталог site/, генератор Zola) должен содержать лендинг, «Что нового» по версиям, гайды со скриншотами, идеи развития, «О проекте», отзывы и правовые страницы на трёх языках (русский – по адресу /, английский – /en/, тайский – /th/). Каждая страница – три файла одинакового устройства; скриншоты – для каждого языка свои.

До переезда сайта на основной домен (этап 6) лендинг и правовые страницы приложения обслуживают продакшен; правки текста выполняются в сайте с блогом.

4.3.16 Требования к ролевой модели Системы

В Системе должна быть реализована ролевая модель участников проекта, приведенная в таблице ниже (Таблица 10). Роль проверяется на сервере, а не скрытием кнопок; порядок ролей: VIEWER < EDITOR < OWNER.

Таблица 10 – Ролевая модель участников проекта

Функция	OWNER	EDITOR	VIEWER	Гость	Ассистент / API-токен
Просмотр данных проекта (обзор, операции, календарь, лимиты, справочники)	да	да	да	да (демо)	право read
Ввод и изменение операций, сверка, перевод описаний	да	да	нет	нет	transactions:write
Удаление операций	да	да	нет	нет	нет (MCP); write (API)
Ведение справочников, курсов, лимитов	да	да	нет	нет	право модуля
Плановые операции, оплата и пропуск платежей	да	да	нет	нет	planning:write; оплата – transactions:write
Архивирование записей справочников	да	да	нет	нет	нет (MCP); write (API)

Функция	OWNER	EDITOR	VIEWER	Гость	Ассистент / API-токен
Переименование проекта, «Сделать семейным», требование 2FA	да	нет	нет	нет	нет
Управление ролями, удаление участников, приглашения	да	нет	нет	нет	нет
Удаление семейного проекта	да	нет	нет	нет	нет
Выход из проекта	да (если есть другой OWNER)	да	да	нет	нет
Профиль, сессии, API-токены, доступ ассистентов	да	да	да	нет	нет

Для каждой роли должны соблюдаться следующие требования:

- OWNER – создатель проекта или участник, назначенный владельцем; в проекте всегда есть хотя бы один OWNER;
- EDITOR – член семьи, ведущий бюджет наравне с владельцем, без права управлять участниками;
- VIEWER – член семьи или иное лицо с правом только просмотра; вверху страниц отображается отметка «Только просмотр»;
- Гость – общий пользователь демо-входа с ролью VIEWER в демо-проекте, без доступа к профилю и настройкам;
- Ассистент и API-токен – действуют от имени пользователя в одном проекте; их права – пересечение роли пользователя и выданных прав.

4.4 Требования к видам обеспечения

4.4.1 Требования к математическому обеспечению

Денежные суммы должны храниться и обрабатываться в десятичной арифметике с фиксированной точностью (тип NUMERIC в СУБД, десятичный тип в приложении); использование чисел с плавающей запятой для денег запрещено. Точность хранения: суммы – NUMERIC(18,4), курсы – не менее 6 знаков после запятой.

Используемые алгоритмы не должны приводить к накоплению ошибок округления:

- сумма позиции – количество × цена, округлённое до точности валюты по правилу «половина вверх», минус скидка;
- скидка на чек распределяется по позициям пропорционально сумме, остаток округления относится к самой крупной позиции;
- округление итогов отчётов должно выполняться после суммирования по точности валюты (этап 1 развития);
- пересчёт в базовую валюту выполняется по курсу на дату операции одной функцией для всех отчётов.

Алгоритмы расчёта бюджета (факт, план, прогноз, доступно), пересчёта лимитов по периодам и расчёта вхождений плановых операций приведены в разделе 4.3 и должны быть покрыты автоматическими тестами.

4.4.2 Требования к информационному обеспечению

4.4.2.1 Требования к составу, структуре и способам организации данных в Системе

Данные Системы должны храниться в СУБД PostgreSQL в базе family_budget; таблицы имеют префикс fb_ и сгруппированы по модулям. Все данные, кроме пользователей, принадлежат проекту (поле project_id). Состав основных таблиц приведен в таблице ниже (Таблица 11).

Таблица 11 – Состав основных таблиц Системы

Модуль	Таблица	Назначение и ключевые поля
Проекты и доступ	fb_users	Пользователи: идентификатор Rauthy (sub, уникален), email, имя, аватар, язык, тема; паролей нет
	fb_projects	Проекты: название, тип PERSONAL/FAMILY, базовая валюта, требование 2FA, автор
	fb_project_members	Участники: проект, пользователь, роль OWNER/EDITOR/VIEWER, дата вступления
	fb_project_invites	Приглашения: роль, срок действия (код не хранится)
	fb_sessions	Сессии: хеш идентификатора, срок, последняя активность, признак второго фактора, IP, браузер, отзыв
	fb_api_tokens	API-токены: хеш, проект, права, срок, признак второго фактора, последнее использование, отзыв
	fb_audit_log	Журнал безопасности: время, пользователь, проект, событие, IP, браузер, подробности (JSONB)
Ассистенты	fb_mcp_grants	Доступ ассистентов: пользователь, клиент, проект, права
Валюты	fb_currencies	Код ISO 4217, название с переводами, символ, знаков после запятой (0–4), архив
Курсы валют	fb_rates	Дата, валюта, курс к базовой, введённое значение, признак обратного ввода
Счета	fb_accounts	Код, название с переводами, тип, валюта, держатель, архив
Категории	fb_categories	Код, название с переводами, тип EXPENSE/INCOME, родитель, порядок, архив
Места	fb_places	Код, название, родитель, варианты написания, тип, адрес, категория и счёт по умолчанию, архив
Люди	fb_people	Код, имя, другие имена, пользователь (для участника), признак удалённого участника, архив
Операции	fb_transactions	Шапка: дата, тип, описание с переводами, автор, план и дата вхождения, счёт, итог, возврат, место

Модуль	Таблица	Назначение и ключевые поля
	fb_transaction_items	Позиции: название, количество, единица, цена, скидка, сумма, категория, примечание, «Для кого»
	fb_postings	Проводки: вид и код ссылки, валюта, сумма со знаком, примечание, позиция
	fb_external_refs	Внешние записи: поставщик, идентификатор, операция, вид связи, время получения
Лимиты	fb_limits	Категория, период WEEK/MONTH/YEAR, сумма (пусто – лимит снят), действует с
Планирование	fb_plans	Название с переводами, тип, счёт, категория, счёт зачисления, сумма, правило, интервал, начало, окончание, активность, серия
	fb_plan_skips	Пропуски вхождений: план, дата, кто пропустил

Вычисляемые показатели не должны храниться: сумма операции по счёту, остаток счёта, курс перевода, курсовая разница, статусы вхождений, все показатели бюджета.

Ограничения целостности: уникальность кода справочника в проекте; уникальность оплаты вхождения (проект, план, дата); один начальный остаток на счёт; сумма проводки не равна нулю; баланс проводок по каждой валюте – отложенным триггером; внешние ключи – только внутри модуля.

Пароли, привязки GitHub и Facebook, настройки второго фактора хранятся сервером входа Rauthy в отдельной базе rauthy.

4.4.2.2 Требования к информационному обмену между модулями Системы

Информационный обмен между модулями Системы должен осуществляться без вмешательства пользователя и без повторного ручного ввода информации: через порты модулей, связанные в модуле сборки приложения. Ссылки между модулями выполняются по коду записи справочника и проверяются сервисом модуля-владельца.

Изменения данных через веб-интерфейс, REST API и MCP должны выполняться одними и теми же сервисами модулей.

4.4.2.3 Требования по использованию классификаторов, унифицированных документов и др.

В Системе должны использоваться:

- ISO 4217 – коды валют и стандартное число знаков после запятой;
- IANA Time Zone Database – часовые пояса пользователей и операций (этап 1);
- ISO 8601 и RFC 3339 – форматы дат и времени в API;
- BCP 47 (ru, en, th) – коды языков.

4.4.2.4 Требования к разработке дополнительных справочников и классификаторов

При создании проекта Система должна заполнять: базовую валюту, набор категорий по умолчанию на трёх языках и счёт «Наличные». Дополнительные классификаторы не разрабатываются.

4.4.2.5 Требования по применению систем управления базами данных

Для хранения данных должна использоваться реляционная СУБД PostgreSQL версии 16. Схема БД должна создаваться и изменяться миграциями каждого модуля (у каждой миграции – прямой и обратный сценарии), применяемыми при запуске приложения.

Приложение и сервер входа должны работать под разными пользователями СУБД без прав суперпользователя, создания баз и ролей (fb_app и rauthy); СУБД доступна только с хоста сервера.

Запросы к СУБД выполняются только параметризованными запросами; формирование SQL склейкой пользовательского ввода запрещено. Пул соединений приложения – 10 соединений; при запуске приложение выполняет до 30 попыток подключения с интервалом 2 секунды.

4.4.2.6 Требования к защите данных от разрушений при авариях и сбоях в электропитании Системы

В Системе должна быть обеспечена защита данных от утраты или нарушения целостности:

- при сбоях электропитания и отказах сервера – средствами СУБД, обеспечивающей сохранность данных на момент последней завершённой транзакции; шапка операции и проводки записываются в одной транзакции СУБД;
- при авариях, приведших к невозможности восстановления данных с сервера, – восстановлением из резервных копий.

4.4.2.7 Требования к контролю, хранению, обновлению и восстановлению данных

Система должна обеспечивать контроль вводимых данных на соответствие формальным правилам (тип, длина, допустимость значений, ссылки на справочники) на сервере независимо от проверок в браузере.

Резервное копирование должно выполняться для обеих баз (family_budget и rauthy):

- ежедневно (в 03:15) и перед каждым обновлением версии;
- в формате pg_dump с шифрованием age; без ключа шифрования копия не создаётся;
- с хранением не более трёх копий каждой базы: самая свежая, давностью 1–6 дней и 7–12 дней;
- с ежемесячной проверкой восстановления во временную базу с подсчётом пользователей и операций.

На этапе 1 развития резервные копии должны дополнительно передаваться на другой сервер или в облачное хранилище.

4.4.2.8 Требования к процедуре придания юридической силы документам, продуцируемым техническими средствами Системы

Требования не предъявляются: Система не формирует документов, имеющих юридическую силу.

4.4.3 Требования к лингвистическому обеспечению

Все экранные формы, сообщения пользователю, письма и пользовательская документация должны быть выполнены на русском, английском и тайском языках. Язык интерфейса определяется в порядке: настройка пользователя → cookie → заголовок Accept-Language браузера → русский; переключатель «RU · EN · TH» расположен внизу бокового меню и в шапке лендинга (адрес /lang/{ru|en|th}).

Строки интерфейса должны храниться в словарях модулей и ядра; поиск ключа – словарь модуля, затем словарь ядра. Полнота переводов каждого модуля должна проверяться автоматическим тестом (templates_are_translated / missing_keys). Названия кнопок и разделов в документации и на сайте – точно как в интерфейсе на этом языке.

Даты – григорианский календарь на всех языках; числа и даты – в формате языка интерфейса.

Исключения из требования трёх языков: данные, вводимые пользователем (показываются в оригинале с отметкой «нет перевода», переводы может дописать ассистент); документация для разработчиков, комментарии в коде и сообщения коммитов – на русском языке; страницы и письма сервера входа Rauthy 0.37 – без тайского языка (устраняется собственным сервисом входа на этапе 6).

Для разработки Системы должны использоваться языки программирования Rust (серверная часть, редакция 2024), JavaScript (небольшой собственный клиентский код без фреймворков), SQL; для вспомогательных скриптов – Python и Bash.

4.4.4 Требования к программному обеспечению

ПО, необходимое для реализации функций Системы, складывается из разрабатываемого прикладного ПО (ППО) и используемого свободного ПО (СПО).

4.4.4.1 Требования к ППО

Разрабатываемое ППО должно отвечать следующим требованиям:

- архитектура – эталон rust-hex-api: Cargo workspace, одна функциональная область – один крейт с доменом, сервисом, портами, адаптерами (REST, веб-страницы, PostgreSQL, память для тестов), миграциями и тестами; связи модулей – в модуле сборки приложения;
- сервер – один исполняемый файл; статические ресурсы (CSS, JS, шрифты, иконки) встроены в исполняемый файл с хешем в имени и кэшем на год; Node.js для запуска не требуется;
- покрытие автоматическими тестами правил домена, изоляции проектов («чужой проект → 404»), «Отменить» в каждом модуле, входа по OIDC, сессий и токенов, CSRF, второго фактора, быстрого ввода и полноты переводов; тесты запускаются без БД на адаптерах в памяти, а на этапе 1 – дополнительно на PostgreSQL;
- описание REST API в формате OpenAPI (Swagger, адрес /docs) в режиме разработки;
- форматирование кода (rustfmt) и отсутствие предупреждений статического анализатора (clippy);
- наличие в репозитории инструкций по запуску, сборке и развёртыванию на русском языке.

4.4.4.2 Требования к СПО

Используемое СПО и библиотеки должны быть общедоступными, с лицензиями, разрешёнными политикой проекта (проверка cargo deny). Перечень СПО приведен в таблице ниже (Таблица 12).

Таблица 12 – Используемое свободное ПО

Слой	ПО	Версия
Язык и сборка	Rust, Cargo	редакция 2024, стабильная версия
Веб-сервер	Axum, Tower	0.8
Шаблоны страниц	Askama	0.16
Доступ к СУБД	sqlx	0.8

Слой	ПО	Версия
Описание API	utoipa (Swagger UI)	актуальная
Вход OIDC	openidconnect	актуальная
MCP-сервер	rmcp	3.5
Стили и частичное обновление	Tailwind CSS, htmx	3, 2
СУБД	PostgreSQL	16
Сервер входа	Rauthy	0.37.1
Обратный прокси	Caddy (сертификаты Let's Encrypt)	актуальная
Контейнеры	Docker, Docker Compose (Podman – для разработки)	актуальная
Базовый образ	Debian bookworm-slim	12
ОС сервера	Ubuntu 24.04 LTS или Debian 12	–
Сайт с блогом	Zola	0.23.6
Шрифты и иконки	Inter, Noto Sans Thai, Lucide	–

На этапах развития допускается применение: Dioxus – для клиентских приложений и пилотного веб-интерфейса (этап 2), SQLite – для хранения данных на устройстве (этап 4).

Для работы пользователей требуется веб-браузер актуальной версии: Google Chrome, Safari (iOS, iPadOS, macOS), Mozilla Firefox, Microsoft Edge. Основные формы Системы должны работать и без JavaScript.

4.4.5 Требования к техническому обеспечению

Для размещения серверной части Системы должен использоваться VDS со следующими характеристиками:

- при использовании готового образа из GHCR – 1–2 виртуальных процессора, 2 ГБ оперативной памяти, 20 ГБ дискового пространства;
- при сборке образа на сервере – не менее 4 ГБ оперативной памяти (или файл подкачки) и дополнительно 10 ГБ дискового пространства.

Образ Системы должен собираться для архитектур linux/amd64 и linux/arm64.

Технические средства пользователей – персональные компьютеры, планшеты и мобильные телефоны с веб-браузером и доступом в сеть Интернет.

4.4.6 Требования к метрологическому обеспечению Системы

Требования к метрологическому обеспечению не предъявляются.

4.4.7 Требования к телекоммуникационному обеспечению Системы

Передача данных по сети Интернет должна осуществляться исключительно по протоколу HTTPS (TLS версии не ниже 1.2) для приложения, сервера входа и MCP-сервера; HTTPS-сертификаты выпускаются автоматически обратным прокси. На сервере открыты только порты 22 (SSH), 80 и 443; СУБД, приложение и Rauthy слушают только локальный интерфейс сервера.

Передача аутентификационных данных в открытом виде запрещена; секреты не передаются в адресной строке и не записываются в журналы.

В Системе должен отсутствовать функционал отправки данных бюджета на почтовые адреса и во внешние хранилища напрямую из интерфейса; данные передаются только ассистентам и клиентам, которым пользователь сам выдал доступ.

4.4.8 Требования к методическому обеспечению Системы

Для пользователей на сайте с блогом должны публиковаться гайды на трёх языках со скриншотами интерфейса на каждом языке: первые шаги, разделы приложения, обзор, операции, календарь, лимиты, категории, счета, места, люди, валюты и курсы, семья, профиль, подключение Claude и ChatGPT. Новая возможность, которой нужно научиться, сопровождается гайдом; при изменении интерфейса гайда скриншоты пересоздаются скриптом.

4.5 Общие технические требования

4.5.1 Требования к численности и квалификации персонала Системы и режиму его работы

Структура и конфигурация Системы должны быть спроектированы с целью минимизации состава обслуживающего персонала: развёртывание, обновление с откатом и резервное копирование выполняются сценариями без ручных операций.

Персонал Системы должен состоять из следующих категорий:

- обслуживающий персонал – администратор (владелец проекта): развёртывание и обновление, настройка Rauthy и SMTP, контроль резервных копий и журналов;
- пользователи Системы – участники проектов.

4.5.1.1 Требования к квалификации персонала

Администратор должен владеть навыками администрирования Linux, Docker и PostgreSQL в объёме инструкции по развёртыванию (docs/deploy-vds.md). Пользователям специальная подготовка не требуется: достаточно навыков работы с веб-браузером; обучение – по гайдам на сайте.

4.5.1.2 Требуемый режим работы персонала

Режим работы пользователей – по необходимости, без ограничений по времени суток. Администратор работает по мере необходимости; о сбоях автоматического обновления администратор уведомляется в Telegram или по email.

4.5.2 Требования к показателям назначения

Основными показателями назначения являются: количество пользователей, число обрабатываемых объектов, время отклика и ограничения нагрузки. Значения показателей являются целевыми для версии 1.0 при размещении на одном VDS с характеристиками раздела 4.4.5 и уточняются по результатам эксплуатации.

4.5.2.1 Количество пользователей и объектов

Значения показателей количества пользователей и объектов приведены в таблице ниже (Таблица 13).

Таблица 13 – Показатели количества пользователей и обрабатываемых объектов

№	Показатель	Значение
1.	Число участников одного проекта	до 10
2.	Число операций, вводимых в одном проекте за день (расчётное / максимальное)	20 / 200
3.	Число проводок в одном проекте, при котором бюджет считается без промежуточных таблиц	до 100 000; при превышении вводится материализованная таблица месячных сумм
4.	Число операций, отображаемых в ленте за месяц	до 5 000 (итоги – по всем операциям)
5.	Число позиций в одной операции	до 100
6.	Число вхождений плановых операций за один расчёт / окно запроса	до 3 700 / до 1 098 дней
7.	Глубина дерева категорий / дерева мест	до 5 / до 3 уровней
8.	Число действующих приглашений на проект	до 20

4.5.2.2 Требования к времени отклика и ограничениям нагрузки

Значения показателей времени отклика и ограничений нагрузки приведены в таблице ниже (Таблица 14).

Таблица 14 – Показатели времени отклика и ограничений нагрузки

№	Показатель	Значение
1.	Время добавления типичного расхода через быстрый ввод (сумма, частая категория, Enter)	не более 5 с
2.	Время отображения первого экрана страницы приложения	не более 1 с
3.	Время отрисовки основного содержимого (LCP) лендинга на телефоне	не более 2,5 с
4.	Объём CSS по сети	не более 50 КБ
5.	Оценка Lighthouse по доступности и производительности («Обзор», «Операции», лендинг)	не менее 90
6.	Максимальный размер тела запроса	64 КБ
7.	Таймаут обработки запроса	15 с (ответ 408)
8.	Число записей на странице REST API	по умолчанию 100, не более 200
9.	Лимит запросов на сессию, API-токен или ассистента	600 в минуту
10.	Лимит запросов публичных страниц с одного IP-адреса	120 в минуту
11.	Таймаут обращения к серверу входа	10 с

4.5.3 Требования к надежности

4.5.3.1 Требования к надежности технических средств и программного обеспечения

Надежность Системы должна достигаться комплексом организационных и технических мер:

- автоматическая проверка каждого изменения в CI (тесты, линтеры, сканеры безопасности) до попадания в основную ветку;
- публикация образа с неизменяемым тегом sha-<коммит> только после прохождения тестов; возможность отката на предыдущую сборку;
- автоматическое обновление с проверкой работоспособности и откатом при сбое, с резервной копией перед обновлением;
- резервное копирование и ежемесячная проверка восстановления;
- ограничение размера журналов контейнеров (5 файлов по 20 МБ) и системного журнала (500 МБ);
- перезапуск контейнеров при сбое средствами контейнерной среды.

На этапе 1 развития должны быть реализованы: проверка готовности с запросом к СУБД, корректное завершение по SIGTERM, хранение состояния «Отменить» и лимитов запросов в БД (для работы нескольких экземпляров), блокировка фоновых задач для одного исполнителя, проверка контейнеров (healthcheck), ограничения прав контейнеров (cap_drop, read_only).

4.5.3.2 Требования к надежности при ошибках пользователя

Система должна обеспечивать корректную обработку ошибочных действий пользователей, неверного формата и недопустимых значений входных данных: показывать сообщение у поля с ошибкой, сохранять введённые данные и позволять исправить ввод. Ошибочное удаление и архивирование отменяются кнопкой «Отменить».

4.5.4 Требования к безопасности

Система не содержит собственных аппаратных средств; требования электробезопасности и пожарной безопасности к оборудованию обеспечиваются поставщиком VDS и производителями устройств пользователей.

4.5.5 Требования к эргономике и технической эстетике

Интерфейс должен быть спокойным и плотным, как у рабочего инструмента: меню слева, одна главная кнопка «Расход», нейтральная палитра; цвет означает смысл (доход, просрочка, превышение лимита), а не украшает.

4.5.5.1 Требования к каркасу интерфейса

Каркас интерфейса должен соответствовать следующим требованиям:

- боковое меню 240 px: логотип, переключатель проекта, кнопка «Расход», разделы «Обзор», «Операции», «Календарь», «Лимиты»; группа «Справочники» – «Категории», «Счета», «Места», «Люди», «Валюты», «Курсы»; внизу – «Проекты и участники», язык, «Профиль», «Выйти»;
- меню сворачивается до иконок (64 px) с подсказками, выбор запоминается на устройстве; при ширине 768–1279 px меню свёрнуто по умолчанию;
- на телефоне (до 768 px) меню скрыто, внизу – панель «Обзор», «Операции», «Календарь», круглая кнопка «+» по центру и «Ещё»;
- над содержимым – заголовок страницы, хлебные крошки на вложенных страницах, действия страницы справа; активный раздел меню подсвечен.

4.5.5.2 Требования к дизайн-системе

Дизайн-система должна определяться токенами, приведенными в таблице ниже (Таблица 15).

Таблица 15 – Токены дизайн-системы

Токен	Значение
Шрифт	Inter и Noto Sans Thai (собственные файлы шрифтов); кегли 12 / 14 / 16 / 20 / 24 / 30; цифры моноширинные
Сетка	Шаг 4 px; отступы 8 / 16 / 24 / 32; ширина содержимого до 1280 px
Цвета	Нейтральная шкала для фона, текста и рамок; один акцент (индиго) для главных действий; доход – зелёный, просрочка и превышение – красный, не менее 80 % лимита – янтарный; суммы расходов – обычным цветом текста
Формы	Радиусы 6 / 8 / 12 px; рамки 1 px; тени только у всплывающих слоёв (меню, диалоги, сообщения)
Иконки	Один набор (Lucide), 20 px, линия 1,5 px; у иконки без подписи – подсказка и aria-label
Тема	Светлая и тёмная: по системной настройке, переключатель в профиле

4.5.5.3 Требования к правилам взаимодействия

Должны соблюдаться следующие правила взаимодействия:

- частые действия – не дальше одного нажатия с любой страницы: добавить расход, переключить проект, перейти к бюджету;
- формы: метки над полями, ошибка под своим полем и сразу, введённое не теряется, сохранение по Enter, первое поле в фокусе;
- удаление и архив – без модальных подтверждений, сообщение 5 секунд с кнопкой «Отменить»; подтверждение – только для необратимого (удаление проекта, выход из проекта);
- горячие клавиши: N – расход, I – доход, T – перевод, Esc – закрыть, ? – список клавиш, Ctrl/Cmd+Enter – сохранить и ещё; клавиши работают и в русской раскладке;
- числа и даты – в формате языка интерфейса, суммы выровнены вправо, валюта рядом с суммой;
- собственные страницы ошибок 404, 403, 429, 500 и 503 на языке пользователя; значок сайта, иконки и манифест PWA.

4.5.5.4 Требования к доступности

Интерфейс должен соответствовать WCAG 2.1 уровня AA: контраст текста не менее 4,5:1, видимый фокус, вся работа с клавиатуры, ссылка «Перейти к содержимому», цели нажатия на телефоне не менее 44 px, ошибки форм связаны с полями (aria-describedby, aria-invalid), сообщения объявляются экранным чтецам (aria-live), учитывается настройка уменьшения анимации.

Частичные обновления страницы выполняются без JS-фреймворков (htmx и небольшой собственный код), основные формы работают и без JavaScript.

4.5.6 Требования к защите информации от несанкционированного доступа

4.5.6.1 Технические требования по защите информации

Требования к защите информации от несанкционированного доступа приведены в таблице ниже (Таблица 16). Каждая мера обязательна и проверяется тестом или сканером перед выпуском версии.

Таблица 16 – Угрозы и меры защиты

Угроза	Меры защиты
Подбор и утечка паролей	Пароли, политика паролей (не менее 10 символов) и защита от подбора – в Rauthy; Система паролей не видит и не хранит
Подмена входа (OIDC)	Authorization code + PKCE S256; проверка state, nonce, подписи ID-токена по JWKS, iss, aud, срока; адрес возврата зафиксирован в Rauthy
Кража сессии	Серверные сессии вместо JWT; идентификатор 256 бит в cookie __Host-session (HttpOnly, Secure, SameSite=Lax), в БД – хеш; неактивность не более 7 дней, жизнь не более 30 дней; новый идентификатор при входе; список устройств и «Выйти везде»
Подделка запросов (CSRF)	SameSite=Lax; CSRF-токен (HMAC от сессии) в каждой форме и в заголовке X-CSRF-Token; проверка заголовков Origin и Referer для изменяющих запросов
Межсайтовый скриптинг (XSS)	Автоэкранирование шаблонов, запрет вывода пользовательских данных без экранирования; CSP без unsafe-inline и unsafe-eval, скрипты и стили – только файлы со своего домена, без CDN
Доступ к чужим данным (IDOR)	Каждый запрос к БД ограничен проектом из контекста доступа; роль проверяется на сервере; тест «чужой проект → 404» в каждом модуле
SQL-инъекции	Только параметризованные запросы
Утечка БД или журналов	Паролей в БД нет; идентификаторы сессий и API-токены – только хеши; секреты – из окружения или файлов секретов; в журналах нет токенов, кодов и cookie
Перегрузка и злоупотребление	Тело запроса не более 64 КБ, таймаут 15 с, страница API не более 200 записей, лимиты запросов на сессию, токен и IP
Перехват трафика	Только HTTPS, HSTS (1 год, includeSubDomains), TLS 1.2+; CORS – только свой домен
Встраивание в чужие сайты	frame-ancestors 'none', X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy с запретом камеры, микрофона, геолокации, платежей и USB; Cross-Origin-Opener-Policy: same-origin
Уязвимые зависимости	cargo audit и cargo deny на каждый запрос на изменение и еженедельно; сканирование образа Trivy; Renovate для обновлений, включая образ Rauthy; контейнер без прав root
Клиенты без браузера	Личные API-токены вместо пароля: права, один проект, срок, показ один раз, хеш в БД, отзыв в профиле
Действия ассистентов	Отдельные права ассистента не выше роли, без удаления; журналирование изменений; инструкция сервера считать данные операций данными, а не командами

4.5.6.2 Требования к механизмам идентификации, аутентификации и авторизации пользователей

Идентификация и аутентификация пользователей веб-интерфейса выполняются сервером входа Rauthy по OpenID Connect; клиентов без браузера – по личному API-токену; ассистентов – по API-токену или OAuth-токену Rauthy. Все способы должны давать модулям один и тот же контекст доступа: пользователь, проект, роль (и права клиента).

4.5.6.3 Требования к средам разработки и тестирования

Разработка и тестирование должны выполняться на стенде разработчика (контейнеры PostgreSQL и Rauthy с демо-пользователями, демо-данные) и в CI; продуктивные данные пользователей в разработке и тестах не используются. Демо-данные и Swagger в продакшене выключены.

Данные реального бюджета, доступные разработчику через MCP при исследовании ошибок, не должны попадать в задачи, запросы на изменение, коммиты, документацию и тесты – в них приводится только описание проблемы на условном примере.

4.5.7 Требования по сохранности информации при авариях

Система должна сохранять данные при следующих аварийных ситуациях:

- программный сбой приложения – данные незавершённых транзакций СУБД откатываются, завершённые сохраняются; паника обработчика перехватывается и возвращает ответ 500 без остановки сервера;
- сбой электропитания или перезагрузка сервера – данные сохраняются средствами СУБД на момент последней завершённой транзакции;
- утрата сервера – восстановление из резервной копии давностью не более суток (после реализации копирования на другой сервер – этап 1).

Целевое время восстановления работоспособности Системы на новом сервере по инструкции развёртывания из резервной копии – не более 4 часов; значение уточняется по результатам ежемесячной проверки восстановления.

4.5.8 Требования к патентной чистоте и лицензионному программному обеспечению

В Системе должно использоваться только СПО с лицензиями, допускающими использование, изменение и распространение. Перечень разрешённых лицензий и запрещённых источников зависимостей должен проверяться автоматически (cargo deny: advisories, bans, licenses, sources). Приобретение лицензий на коммерческое ПО не требуется.

4.5.9 Требования по стандартизации и унификации

Модули Системы должны строиться по единой эталонной архитектуре (rust-hex-api): одинаковая структура крейта, одинаковые адаптеры, единые механизмы ошибок, доступа, многоязычности и «Отменить» из ядра. Справочники должны иметь единые поля и поведение (раздел 4.3).

4.5.10 Требования к разработке

Разработка должна вестись в репозитории проекта по следующим правилам:

- каждое изменение – отдельная ветка и запрос на изменение в основную ветку; прямые изменения основной ветки не допускаются;

- перед запросом на изменение выполняются тесты (make test) и линтеры (make lint: rustfmt, clippy без предупреждений, shellcheck, actionlint, hadolint, ruff); при изменении шаблонов – пересборка CSS (make css); при изменении сайта – make site-check и make site;
- всё, что видит пользователь, реализуется сразу на трёх языках в том же запросе на изменение;
- изменение, заметное пользователю, сопровождается записью в «Что нового»; новая возможность – гайдом; задуманное, но отложенное – идеей на сайте;
- текст коммитов, документации для разработчиков и комментариев в коде – на русском языке;
- регрессионный тест на каждую исправленную ошибку.

4.5.11 Требования к регистрации событий

Система должна регистрировать события безопасности в журнале безопасности (раздел 8.5), а технические события – в журнале приложения: запуск, повторы подключения к СУБД, выполнение фоновых задач, ошибки обработки запросов. Журнал приложения не должен содержать токенов, кодов второго фактора, cookie и паролей. На этапе 1 развития журнал приложения должен вестись в структурированном виде (JSON) с идентификатором запроса.

5 Состав и содержание работ

Состав и содержание работ по разработке Системы определяются в соответствии с таблицей ниже (Таблица 17). Сроки этапов 2–6 определяются при планировании каждого этапа; переход к следующему этапу допускается, когда в отчётах нет известных ошибок.

Таблица 17 – Состав и содержание работ

Наименование этапа	**Содержание работ**	**Сроки выполнения**	**Результаты работ / отчётная документация**
Этап 0. Цифрам можно верить (версия 0.1.x)	Исправление ошибок отчётов по результатам аудита: лимиты по периодам, тип категории, суммы без курса, правка плана задним числом, проверка оплаты плана, точность валюты, уникальность начального остатка; закрытие уязвимостей: токены без 2FA, роли, приглашения вместо добавления по email, гостевой вход, ошибки входа; регрессионные тесты	Выполнен	Версия 0.1.x; записи «Что нового»; отметки в дорожной карте
Этап 1. Фундамент для роста (версия 0.2)	Тесты на PostgreSQL в CI; структурированные журналы, проверка готовности, корректное завершение; «Отменить» и лимиты запросов в БД, версии записей, лента изменений; атомарные создание и удаление проекта; часовой пояс пользователя и операции; округление после суммирования; копии на другой сервер; ограничения контейнеров; ядро без сервера; модель прав клиентов (Actor); приём операций без дублей	Ориентировочно 2–3 недели	Версия 0.2; статья о выпуске; обновлённые README и инструкция по развёртыванию
Этап 2. Удобно каждый день (версия 0.3)	Сообщения проверок на трёх языках во всех модулях; автоматические курсы; поиск и фильтры операций, постраничный вывод; генерация кодов из названия; мастер первого запуска; буддийский календарь; пилот интерфейса на Dioxus	Определяется при планировании	Версия 0.3; гайды по новым функциям
Этап 3. Приложение работает за вас (версия 0.4)	Импорт и «Входящие»: SMS, уведомления, чеки и выписки банков; «Быстрые команды» iPhone; Telegram-бот и Mini App; напоминания; импорт CSV и экспорт; развитие MCP; автопроведение планов	Определяется при планировании	Версия 0.4; ТЗ «Платформы» (этапы F, G)

Наименование этапа	Содержание работ	Сроки выполнения	Результаты работ / отчётная документация
Этап 4. Везде и без интернета (версия 0.6)	Синхронизация с очередью на устройстве; приложения для iPhone, Android и компьютера с работой без сети; локальный MCP; push-напоминания	Определяется при планировании	Версия 0.6; приложения в магазинах; ТЗ «Платформы» (этапы H–K)
Этап 5. Аналитика и семья (версия 0.7)	Графики и капитал; покупка в другой валюте; кредитные карты, долги, взаиморасчёты; перенос остатка лимита и цели; месяц с дня зарплаты; теги; фото чека	Определяется при планировании	Версия 0.7
Этап 6. Запуск для людей (версия 1.0)	Собственный сервис входа (тайский язык, TOTP, passkey, Telegram, корректный выход); сайт на основном домене; отзывы на сайте; Lighthouse, Mozilla Observatory, формальная проверка OWASP ASVS уровня 2; другие ассистенты по спросу; вложенные проекты	Определяется при планировании	Версия 1.0; протокол проверки безопасности и доступности

5.1 Порядок контроля и приемки выполнения работ

5.1.1 Виды, состав, объем и методы испытаний Системы и ее составных частей

Испытания Системы должны проводиться при каждом изменении (автоматически) и перед каждым выпуском версии (автоматически и вручную).

Должны проводиться следующие виды испытаний:

- автоматические испытания в CI на каждый запрос на изменение: модульные тесты домена, тесты сервисов и веб-адаптеров на адаптерах в памяти, тесты на PostgreSQL 16, форматирование и статический анализ, проверка зависимостей (cargo audit, cargo deny), сканирование образа (Trivy), динамическое сканирование OWASP ZAP baseline, проверка сценариев оболочки, файлов CI и Dockerfile, проверка наличия записи «Что нового»;
- еженедельное сканирование опубликованного образа (cargo audit, cargo deny, Trivy);
- проверка сайта: одинаковое устройство трёх языковых версий страниц, ссылки, скриншоты для каждого языка, ключи переводов, сборка и проверка ссылок;
- ручная проверка в браузере каждой изменённой страницы на русском, английском и тайском языках, на компьютере и телефоне, в светлой и тёмной теме;
- опытная эксплуатация каждой функции на реальной семье до выпуска версии;
- приемочная проверка версии 1.0: Lighthouse, Mozilla Observatory, чек-лист OWASP ASVS уровня 2, проверка стенда Rauthy.

Обязательные автоматические проверки для каждого модуля: правила домена, изоляция проектов («чужой проект → 404»), роль VIEWER (403 на изменение), «Отменить», полнота переводов шаблонов.

5.1.2 Общие требования к приемке работ

Приемка версии выполняется Заказчиком по результатам автоматических проверок и ручной проверки. Версия принимается, если выполнены критерии приемки, приведенные ниже, и в отчётах нет известных ошибок.

Критерии приемки:

- пользователь без членства в проекте не видит его данных ни в интерфейсе, ни в REST API, ни через ассистента;
- VIEWER получает 403 на любое изменение и не видит кнопок изменения;
- операцию, у которой сумма проводок по какой-либо валюте не равна нулю, нельзя сохранить ни через API, ни прямой записью в БД; проводка на категорию другого типа отклоняется;
- обмен 10 000 ₽ → 3 912,40 ฿ с комиссией 50 ₽ – одна операция: остатки обоих счетов верны, расходы в бюджете выросли только на 50 ₽, доходы не изменились; удаление или «Отменить» возвращает оба остатка;
- остаток счёта одинаков в выписке, справочнике счетов и «Обзоре»; валюту счёта с проводками изменить нельзя;
- категорию нельзя перенести в собственного потомка, глубина дерева не более 5 уровней; тип категории с операциями не меняется;
- аренда 5-го числа видна в календаре на 12 месяцев вперёд; «Оплатил» создаёт одну операцию, повторная оплата отклоняется, удаление операции возвращает «Ожидается»;
- месячный лимит в годовом бюджете учитывается за 12 месяцев; расход в «Кафе» уменьшает «Доступно» у «Еды»;
- суммы в валюте без курса не входят в итоги и сопровождаются пометкой «нет курса»;
- описание, введённое на тайском языке, показывается на русском после добавления перевода, а до него – в оригинале с отметкой;
- типичный расход (сумма, частая категория, Enter) добавляется с любой страницы за время не более 5 секунд без перехода; «Отменить» в сообщении его удаляет;
- выбор места в быстром вводе подставляет категорию и счёт места или его сети;
- фильтр «Для кого» показывает расходы на выбранного человека по позициям чека;
- одно SMS банка никогда не создаёт двух операций; ассистент дополняет операцию из SMS вместо создания второй;
- ассистент не может удалить или архивировать данные; список его инструментов соответствует выданным правам;
- на компьютере меню слева и сворачивается, на телефоне – нижняя панель с «+»; вся навигация доступна с клавиатуры;
- гость на / видит лендинг на языке браузера, вошедший пользователь попадает на «Обзор»;
- первый вход через Rauthy (пароль, GitHub или Facebook) создаёт пользователя с личным проектом, повторный находит его же; аккаунт Facebook с email существующего пользователя не получает доступа к его данным без привязки;
- в проекте с «Требовать 2FA» участник без второго фактора не видит данных проекта;

- Система не хранит паролей; «Выйти» завершает сессию Системы и сессию Rauthy;
- Lighthouse не менее 90 по доступности и производительности на лендинге, «Обзоре» и «Операциях» (к версии 1.0).

5.1.3 Порядок выполнения доработок и устранения ошибок, выявленных в процессе испытаний и эксплуатации

Ошибки, выявленные при испытаниях и эксплуатации, должны регистрироваться задачами в репозитории проекта с описанием проблемы на условном примере (без данных реального бюджета).

Исправление ошибки должно сопровождаться регрессионным тестом, воспроизводящим ошибку, и записью в «Что нового». Ошибки, из-за которых отчёты показывают неверные числа, исправляются в первую очередь: до их исправления новые функции не выпускаются.

Документация (README, инструкции, гайды) должна актуализироваться в том же запросе на изменение, что и код.

5.1.4 Сведения о порядке учета и контроля выполняемых работ

Учёт работ ведётся в репозитории проекта: задачи, запросы на изменение, отметки выполнения пунктов дорожной карты (docs/roadmap.md), реестр технического долга (docs/tech-debt.md), записи «Что нового» по версиям. Приемку версий выполняет Заказчик.

6 Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу Системы в действие

6.1 Развертывание и конфигурирование

Для ввода Системы в действие Заказчик (администратор) должен:

- арендовать VDS с характеристиками раздела 4.4.5 и зарегистрировать домен с поддоменом сервера входа (auth.<домен>);
- выполнить установку сценарием развёртывания (scripts/deploy-vds.sh install с файлом настроек по образцу deploy/vds.conf.example) либо вручную по инструкции docs/deploy-vds.md: Docker, обратный прокси Caddy с HTTPS, межсетевой экран, ограничение журналов, PostgreSQL с пользователями без суперправ, Rauthy, приложение из готового образа GHCR;
- задать секреты в окружении или файлах секретов: секрет сессии (не короче 32 символов), секрет клиента Rauthy, ключ администратора Rauthy, параметры SMTP, ключи GitHub и Facebook (в админке Rauthy), ключ шифрования резервных копий;
- настроить клиентов Rauthy: family-budget (адрес возврата https://<домен>/auth/callback), claude и chatgpt (для OAuth ассистентов); включить email-коды второго фактора; админку Rauthy защитить passkey;
- включить резервное копирование, проверку восстановления и автоматическое обновление с уведомлениями.

Приложение в продакшене должно запускаться с параметрами APP_ENV=prod, SEED_DEMO=0, без Swagger; без обязательных секретов и HTTPS приложение не запускается.

Разработка на продуктивном сервере не ведётся; для разработки и проверки используется стенд в контейнерах (make, make dev).

6.2 Приведение поступающей в Систему информации к виду, пригодному для обработки с помощью ЭВМ

При создании проекта Система должна автоматически создавать базовую валюту, категории по умолчанию на трёх языках и счёт «Наличные». Пользователь заводит счета с начальными остатками, валюты и курсы; дальнейшая информация поступает в Систему вводом операций в веб-интерфейсе, через ассистентов и внешние клиенты.

На этапе 2 развития должен быть реализован мастер первого запуска (валюты семьи с курсами, счета, первые регулярные платежи), на этапе 3 – импорт банковских выписок и CSV с поиском дублей.

6.3 Изменения, которые необходимо осуществить в объекте автоматизации

6.3.1 Создание условий функционирования объекта автоматизации

Для функционирования Системы пользователям необходимы устройства с веб-браузером актуальной версии и доступом в сеть Интернет; для работы через ассистента – подключённый Claude или ChatGPT. Изменение иных условий не требуется.

6.3.2 Создание необходимых для функционирования Системы подразделений и служб

Создание подразделений и служб не требуется. Функции администратора выполняет владелец проекта.

6.3.3 Сроки и порядок обучения пользователей

Обучение пользователей выполняется самостоятельно по гайдам на сайте с блогом до начала работы с соответствующей функцией. Приглашённый участник может ознакомиться с Системой в демо-проекте.

7 Требования к документированию

Документация на Систему должна вестись в репозитории проекта в формате Markdown и на сайте с блогом и должна учитывать рекомендации:

- ГОСТ Р 59853-2021 – в части терминологии;
- ГОСТ 34.602-2020 – в части состава и содержания технического задания и настоящего ЧТЗ;
- ГОСТ Р 59792-2021 – в части видов испытаний.

Документация для разработчиков ведётся на русском языке; пользовательская документация (гайды, «Что нового», правовые страницы) – на русском, английском и тайском языках.

7.1 Состав документации

В состав документации на Систему входят документы, приведенные в таблице ниже (Таблица 18).

Таблица 18 – Состав документации

№	Наименование документа	Краткое содержание
1	Частное техническое задание (настоящий документ)	Функциональные и нефункциональные требования к Системе и этапам её развития
2	Техническое задание «Семейная бухгалтерия», версия 3	Исходные требования к Системе; docs/tz-v3.md – сведения о реализации
3	Дорожная карта (docs/roadmap.md)	Результаты аудита, этапы развития с отметками выполнения
4	Технические задания на развитие (docs/group-1.md, docs/platforms.md, docs/mcp-plan.md, docs/id-service-prompt.md)	Карточка операции; Telegram, приложения и офлайн; MCP-коннектор; собственный сервис входа
5	Описание Системы и команд (README.md)	Назначение, стек, запуск, структура, тесты, безопасность
6	Правила разработки (CLAUDE.md)	Три языка, «Что нового», работа с данными MCP, проверки перед запросом на изменение
7	Руководство администратора (docs/deploy-vds.md)	Развёртывание на VDS, HTTPS, Rauthy, резервные копии, обновление и откат
8	Подключение ассистентов (docs/mcp.md)	Настройка MCP-сервера, клиентов Rauthy, подключение Claude и ChatGPT
9	Схема БД (docs/db-schema.drawio)	Таблицы и связи по модулям
10	Описание REST API (Swagger, /docs)	Методы, параметры и схемы REST API
11	Реестр технического долга (docs/tech-debt.md), внешние записи (docs/external-refs.md)	Известные ограничения и решения
12	Руководство пользователя (гайды на сайте)	Пошаговые инструкции со скриншотами на трёх языках
13	«Что нового»	Описание изменений каждой версии на трёх языках
14	Правовые страницы	Политика конфиденциальности, условия использования, удаление данных на трёх языках

8 Требования к информационной безопасности

8.1 Требования к реализации ролевой модели

Пользователю должны предоставляться минимально необходимые для его задач права: роль в проекте (раздел 4.3, Таблица 10), права API-токена и права ассистента – не выше роли. Работа пользователей должна осуществляться под уникальными персонифицированными учётными записями; исключение – общий гостевой пользователь демо-проекта с правом только просмотра.

Функции администрирования сервера, СУБД и сервера входа не должны быть доступны через интерфейс Системы. Админка Rauthy доступна только администратору с обязательным passkey.

Права ассистента настраиваются только пользователем в браузере; ассистент не может изменить свои права.

8.2 Требования к реализации парольной политики

Пароли пользователей, политика паролей (длина не менее 10 символов) и защита от подбора реализуются сервером входа Rauthy; Система паролей не получает и не хранит. Хранение паролей в Rauthy – стойкими алгоритмами хеширования с солью средствами Rauthy.

8.3 Требования к механизмам идентификации и аутентификации

Вход пользователей – через Rauthy по OpenID Connect (authorization code + PKCE S256) с проверкой ID-токена; второй фактор – 6-значный код по email или passkey; владелец проекта может потребовать второй фактор от участников.

Сессии Системы – серверные, с ограничением неактивности (7 дней) и срока жизни (30 дней), с отзывом отдельной сессии и «Выйти везде». Гостевая сессия – 24 часа.

Клиенты без браузера – личные API-токены с правами, проектом и сроком; ассистенты – API-токен или OAuth-токен Rauthy с проверкой подписи, издателя, получателя и срока.

На этапе 6 должен быть внедрён собственный сервис входа с тайским языком, TOTP, passkey, входом через Telegram и корректным выходом.

8.4 Требования к защите каналов передачи данных

Передача данных должна осуществляться исключительно по HTTPS с TLS версии не ниже 1.2 и HSTS (1 год, includeSubDomains). CORS разрешается только для собственного домена: методы GET, POST, PUT, DELETE; заголовки Authorization, Content-Type, X-CSRF-Token, X-Project-Id. В продакшене приложение не запускается с адресом приложения или сервера входа без HTTPS.

8.5 Требования к аудиту

8.5.1 Состав регистрируемых событий

В журнале безопасности Системы подлежат обязательной регистрации следующие события:

- вход и выход: успешный вход (login), неуспешный вход (login_failed), выход (logout), «Выйти везде» (logout_all), отзыв сессии (session_revoked);
- проекты и участники: создание и удаление проекта, перевод в семейный, изменение требования 2FA, изменение роли, удаление и вступление участника, создание и отзыв приглашения;
- API-токены: выпуск и отзыв;

- ассистенты: открытие и отключение доступа, каждое изменение данных через ассистента (mcp_write);
- удаление аккаунта.

Для каждого события фиксируются: время, пользователь, проект, IP-адрес, браузер и подробности в формате JSON. События входа и второго фактора дополнительно ведёт Rauthy.

8.5.2 Разграничение доступа к данным аудита

Пользователь видит только собственные события журнала безопасности. Гость журнал не видит. Полный журнал доступен только администратору средствами СУБД.

8.5.3 Функциональные требования к экранной форме

Журнал безопасности в разделе «Профиль → Безопасность» должен отображать последние 100 событий пользователя в таблице с колонками: Дата, Событие (название на языке интерфейса), Проект, IP. На этапах развития должны быть добавлены фильтры по периоду, событию и проекту и постраничный вывод.

8.5.4 Хранение данных аудита

Записи журнала безопасности должны храниться 365 дней; более старые записи удаляются фоновой задачей каждые 6 часов. При удалении аккаунта записи пользователя обезличиваются: удаляются ссылка на пользователя, IP-адрес и сведения о браузере.

8.6 Требования к защите приложений (контейнеризация)

Для Системы, работающей в контейнерах Docker, устанавливаются требования:

- запуск приложения в контейнере от непривилегированного пользователя (не root);
- передача секретов только через переменные окружения или файлы секретов (*_FILE), без включения в образ и репозиторий; в продакшене приложение не запускается со значениями секретов по умолчанию;
- использование образов только из GHCR проекта и официальных образов зависимостей; образ сопровождается подписанной аттестацией происхождения, проверяемой командой gh attestation verify;
- обновление базового образа с исправлениями безопасности при каждой сборке; сканирование образа Trivy;
- порты СУБД, Rauthy и приложения доступны только с локального интерфейса сервера;
- на этапе 1: удаление лишних привилегий (cap_drop), файловая система только для чтения, проверка состояния контейнера, проверка аттестации при автоматическом обновлении.

8.7 Требования к защите API и MCP

Для REST API и MCP-сервера устанавливаются требования:

- доступ только по личному API-токену (заголовок Authorization: Bearer) или, для MCP, по OAuth-токену Rauthy; API-токен не принимается на адресах вне /api и /mcp;
- проект запроса определяется токеном; запрос к данным другого проекта завершается ответом 404;
- лимиты запросов, размер тела и таймаут – по таблице показателей (Таблица 14);

- ответы REST API об ошибках – в формате JSON без технических подробностей;
- MCP-сервер не предоставляет инструментов удаления и архивирования; данные, полученные от пользователей (описания, названия, заметки), передаются ассистенту как данные и не должны исполняться как инструкции.

8.8 Требования к защите персональных данных

Система должна обрабатывать минимально необходимые персональные данные: email, имя, аватар (из Rauthy), IP-адрес и сведения о браузере (в сессиях и журнале безопасности). Сторонние трекеры и счётчики не используются; статистика воронки ведётся без IP-адресов и cookie.

Пользователь должен иметь возможность удалить аккаунт: удаляются личные данные в Системе и пользователь в Rauthy, записи журнала обезличиваются, семейные проекты остаются у других участников. Порядок удаления данных описывается на странице «Удаление данных» (/data-deletion) на трёх языках.

Данные бюджета пользователя, доступные разработчику через MCP при исследовании ошибок, не должны попадать в задачи, запросы на изменение, коммиты, документацию и тесты.